DÉMO · CHANGEMENTS ENTRE LES ÉTAPES

Projet Car Catalog

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `− ligne retirée` `@@ position dans le fichier`

1. **Étape 1 → 2** · etape-2-api-liste (10 fichiers)
2. **Étape 2 → 3** · etape-3-details (2 fichiers)
3. **Étape 3 → 4** · etape-4-favoris (8 fichiers)
4. **Étape 4 → 5** · etape-5-login (5 fichiers)
5. **Étape 5 → 6** · etape-6-routes-protegees (2 fichiers)
6. **Étape 6 → 7** · etape-7-error-boundary (4 fichiers)
7. **Étape 7 → 8** · etape-8-tests (5 fichiers)
8. **Étape 8 → 9** · etape-9-architecture (107 fichiers)
9. **Étape 9 → 10** · etape-10-ia (4 fichiers)

Étape 1 → 2 etape-1-template → etape-2-api-liste

La liste vient de l'API : axios, React Query (useQuery), proxy Vite, états loading / error, 6 voitures

MODIFIÉ **package.json** +3 −0

```
@@ -9,6 +9,9 @@
    "preview": "vite preview"
  },
  "dependencies": {
+   "@tanstack/react-query": "^5.90.0",
+   "@tanstack/react-query-devtools": "^5.90.0",
+   "axios": "^1.13.0",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.11.0"
```

AJOUTÉ **src/api/actions.js** +4 −0

```
@@ -1,0 +1,4 @@
+ import axiosQuery from "../lib/axiosQuery";
+
+ // Liste des voitures d'une année (la page Home)
+ export const fetchCarsByYear = (year) => axiosQuery("get", `/api/cars/year/${year}`);
```

AJOUTÉ **src/api/config.js** +2 −0

```
@@ -1,0 +1,2 @@
+ // Les requêtes passent par le proxy Vite (/api) : pas besoin d'URL complète en dev
+ export const BASE_URL = "";
```

AJOUTÉ **src/components/Loading.jsx** +9 −0

```
@@ -1,0 +1,9 @@
+ // Le chargement : un spinner et un texte, réutilisé par toutes les pages
+ export default function Loading({ text = "Loading..." }) {
+   return (
+     <div className="center-page">
+       <div className="spinner"></div>
+       <p>{text}</p>
+     </div>
+   );
```

```
+ }
```

AJOUTÉ `src/lib/axiosQuery.js` +20 −0

```
@@ -1,0 +1,20 @@
+ import axios from "axios";
+ import { BASE_URL } from "../api/config";
+
+ const axiosInstance = axios.create({
+   baseURL: BASE_URL,
+   headers: {
+     "Content-Type": "application/json",
+     Accept: "application/json",
+   },
+   timeout: 30000,
+ });
+
+ // Un seul point d'entrée pour tous les appels API : on récupère directement les données
+ const axiosQuery = async (method, url, data) => {
+   const response = method === "post" ? await axiosInstance.post(url, data) : await axiosInstance.get(url);
+   return response.data;
+ };
+
+ export { axiosInstance };
+ export default axiosQuery;
```

AJOUTÉ `src/lib/queryClient.js` +10 −0

```
@@ -1,0 +1,10 @@
+ import { QueryClient } from "@tanstack/react-query";
+
+ export const queryClient = new QueryClient({
+   defaultOptions: {
+     queries: {
+       retry: 3,
+       refetchOnWindowFocus: true,
+     },
+   },
+ });
```

MODIFIÉ `src/main.jsx` +7 −1

```
@@ -1,10 +1,16 @@
  import { StrictMode } from "react";
  import { createRoot } from "react-dom/client";
+ import { QueryClientProvider } from "@tanstack/react-query";
+ import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
+ import { queryClient } from "./lib/queryClient";
  import App from "./App";
  import "./styles.css";

  createRoot(document.getElementById("root")).render(
    <StrictMode>
-     <App />
+     <QueryClientProvider client={queryClient}>
+       <App />
+       <ReactQueryDevtools initialIsOpen={false} />
+     </QueryClientProvider>
    </StrictMode>
  );
```

MODIFIÉ `src/pages/HomePage.jsx` +21 −1

```
@@ -1,7 +1,27 @@
```

```
1  - import cars from "../data/cars";
   1 + import { useQuery } from "@tanstack/react-query";
   2 + import { fetchCarsByYear } from "../api/actions";
2  3   import CarCard from "../components/CarCard";
   4 + import Loading from "../components/Loading";
3  5
4  6   export default function HomePage() {
   7 +   const { data, isLoading, error } = useQuery({
   8 +     queryKey: ["cars", 2012],
   9 +     queryFn: () => fetchCarsByYear(2012),
  10 +   });
  11 +
  12 +   if (isLoading) return <Loading text="Loading cars..." />;
  13 +
  14 +   if (error) {
  15 +     return (
  16 +       <div className="center-page">
  17 +         <p className="error-text">Error loading cars: {error.message}</p>
  18 +       </div>
  19 +     );
  20 +   }
  21 +
  22 +   // On limite l'affichage à 6 voitures
  23 +   const cars = (data?.Cars || []).slice(0, 6);
  24 +
5  25     return (
6  26       <div className="container">
7  27         <h1 className="page-title">Car Catalog</h1>
```

MODIFIÉ **src/styles.css** +4 −0

```
@@ -177,6 +177,10 @@
177 177   .label,
178 178   .detail-label {
179 179     color: var(--text-secondary);
    180 + }
    181 +
    182 + .error-text {
    183 +   color: var(--error);
180 184   }
181 185
182 186   .subtitle {
```

MODIFIÉ **vite.config.js** +9 −0

```
@@ -3,4 +3,13 @@
3  3
4  4   export default defineConfig({
5  5     plugins: [react()],
   6 +   server: {
   7 +     // Le proxy évite les erreurs CORS en dev : /api/... est envoyé à https://myfakeapi.com/api/...
   8 +     proxy: {
   9 +       "/api/": {
  10 +         target: "https://myfakeapi.com",
  11 +         changeOrigin: true,
  12 +       },
  13 +     },
  14 +   },
6  15  });
```

Étape 2 → 3 etape-2-api-liste → etape-3-details

La page de détails vient de l'API : fetchCarById, useQuery avec ["car", id]

MODIFIÉ `src/api/actions.js` +3 −0

```
@@ -2,3 +2,6 @@
 
  // Liste des voitures d'une année (la page Home)
  export const fetchCarsByYear = (year) => axiosQuery("get", `/api/cars/year/${year}`);
+
+ // Détails d'une voiture (la page /car/:id)
+ export const fetchCarById = (id) => axiosQuery("get", `/api/cars/${id}`);
```

MODIFIÉ `src/pages/CarDetailPage.jsx` +14 −3

```
@@ -1,13 +1,24 @@
+ import { useQuery } from "@tanstack/react-query";
  import { useParams, Link } from "react-router-dom";
- import cars from "../data/cars";
+ import { fetchCarById } from "../api/actions";
  import { BackIcon, HeartIcon } from "../components/Icons";
+ import Loading from "../components/Loading";
  import { getColorHex } from "../utils/colorUtils";
 
  export default function CarDetailPage() {
    const { id } = useParams();
-   const car = cars.find((c) => c.id === Number(id));
 
-   if (!car) {
+   // L'id fait partie de la clé : chaque voiture a son propre cache
+   const { data, isLoading, error } = useQuery({
+     queryKey: ["car", id],
+     queryFn: () => fetchCarById(id),
+   });
+
+   const car = data?.Car;
+
+   if (isLoading) return <Loading text="Loading car details..." />;
+
+   if (error || !car) {
      return (
        <div className="container">
          <p>Car not found</p>
```

Étape 3 → 4 etape-3-details → etape-4-favoris

Les favoris avec Redux Toolkit : favoritesSlice (toggleFavorite), store, Provider, Home / Détail / Favoris

MODIFIÉ `package.json` +2 −0

```
@@ -9,11 +9,13 @@
      "preview": "vite preview"
    },
    "dependencies": {
+     "@reduxjs/toolkit": "^2.11.2",
      "@tanstack/react-query": "^5.90.0",
      "@tanstack/react-query-devtools": "^5.90.0",
      "axios": "^1.13.0",
      "react": "^19.2.0",
      "react-dom": "^19.2.0",
+     "react-redux": "^9.2.0",
      "react-router-dom": "^7.11.0"
    },
    "devDependencies": {
```

SUPPRIMÉ `src/data/cars.js` +0 −12

```
@@ -1,12 +1,0 @@
- // Données en dur, au même format que l'API (myfakeapi.com) :
- // à l'étape 2, on les remplace par un appel API.
- const cars = [
-   { id: 1, car: "Tesla", car_model: "Model S", car_color: "Red", car_model_year: 2024, car_vin: "5YJSA1E26MF168934", price: "$79,990", availability: true },
-   { id: 2, car: "BMW", car_model: "i8", car_color: "Blue", car_model_year: 2023, car_vin: "WBY2Z2C54GV676451", price: "$147,500", availability: false },
-   { id: 3, car: "Audi", car_model: "e-tron", car_color: "Green", car_model_year: 2024, car_vin: "WA1LAAGE7MB012345", price: "$65,900", availability: true },
-   { id: 4, car: "Porsche", car_model: "Taycan", car_color: "Orange", car_model_year: 2023, car_vin: "WP0AB2Y14NSA12345", price: "$82,700", availability: true },
-   { id: 5, car: "Mercedes", car_model: "EQS", car_color: "Purple", car_model_year: 2024, car_vin: "W1KCG4EB5PA012345", price: "$102,310", availability: false },
-   { id: 6, car: "Lucid", car_model: "Air", car_color: "Turquoise", car_model_year: 2023, car_vin: "50EA1TEA7PA012345", price: "$77,400", availability: true },
- ];
-
- export default cars;
```

MODIFIÉ `src/main.jsx` +8 −4

```
@@ -1,16 +1,20 @@
  import { StrictMode } from "react";
  import { createRoot } from "react-dom/client";
+ import { Provider } from "react-redux";
  import { QueryClientProvider } from "@tanstack/react-query";
  import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
  import { queryClient } from "./lib/queryClient";
+ import { store } from "./store/store";
  import App from "./App";
  import "./styles.css";

  createRoot(document.getElementById("root")).render(
    <StrictMode>
-     <QueryClientProvider client={queryClient}>
-       <App />
-       <ReactQueryDevtools initialIsOpen={false} />
-     </QueryClientProvider>
+     <Provider store={store}>
```

```
      14 +       <QueryClientProvider client={queryClient}>
      15 +         <App />
      16 +         <ReactQueryDevtools initialIsOpen={false} />
      17 +       </QueryClientProvider>
      18 +     </Provider>
15    19      </StrictMode>
16    20    );
```

MODIFIÉ **src/pages/CarDetailPage.jsx** +7 −2

```
@@ -1,12 +1,16 @@
1     1     import { useQuery } from "@tanstack/react-query";
2     2     import { useParams, Link } from "react-router-dom";
      3 + import { useDispatch, useSelector } from "react-redux";
3     4     import { fetchCarById } from "../api/actions";
      5 + import { selectFavorites, toggleFavorite } from "../store/favoritesSlice";
4     6     import { BackIcon, HeartIcon } from "../components/Icons";
5     7     import Loading from "../components/Loading";
6     8     import { getColorHex } from "../utils/colorUtils";
7     9
8     10    export default function CarDetailPage() {
9     11      const { id } = useParams();
      12 +   const dispatch = useDispatch();
      13 +   const favorites = useSelector(selectFavorites);
10    14
11    15      // L'id fait partie de la clé : chaque voiture a son propre cache
12    16      const { data, isLoading, error } = useQuery({
@@ -15,6 +19,7 @@
15    19      });
16    20
17    21      const car = data?.Car;
      22 +   const isFavorite = car ? favorites.some((fav) => fav.id === car.id) : false;
18    23
19    24      if (isLoading) return <Loading text="Loading car details..." />;
20    25
@@ -38,8 +43,8 @@
38    43              <h1 className="detail-title">
39    44                {car.car} {car.car_model}
40    45              </h1>
41        -           <button className="favorite-btn">
42        -             <HeartIcon />
      46 +           <button className={`favorite-btn ${isFavorite ? "favorite-active" : ""}`} onClick
          ={() => dispatch(toggleFavorite(car))}>
      47 +             <HeartIcon filled={isFavorite} />
43    48              </button>
44    49            </div>
45    50
```

MODIFIÉ **src/pages/FavoritesPage.jsx** +14 −9

```
@@ -1,18 +1,23 @@
1         - import cars from "../data/cars";
      1 + import { useDispatch, useSelector } from "react-redux";
2     2     import CarCard from "../components/CarCard";
3         -
4         - // Pour l'instant, des favoris en dur (à l'étape 4 : Redux Toolkit)
5         - const favorites = cars.filter((car) => [1, 3, 4].includes(car.id));
      3 + import { selectFavorites, toggleFavorite } from "../store/favoritesSlice";
6     4
7     5     export default function FavoritesPage() {
      6 +   const dispatch = useDispatch();
      7 +   const favorites = useSelector(selectFavorites);
      8 +
8     9       return (
```

```
 9  10        <div className="container">
10  11          <h1 className="page-title">My Favorites</h1>
11      -       <div className="car-grid">
12      -         {favorites.map((car) => (
13      -           <CarCard key={car.id} car={car} isFavorite />
14      -         ))}
15      -       </div>
    12  +       {favorites.length === 0 ? (
    13  +         <p className="subtitle">No favorites yet. Add some cars from the home page!</p>
    14  +       ) : (
    15  +         <div className="car-grid">
    16  +           {favorites.map((car) => (
    17  +             <CarCard key={car.id} car={car} isFavorite onToggleFavorite={(c) => dispatch(to
        ggleFavorite(c))} />
    18  +           ))}
    19  +         </div>
    20  +       )}
16  21        </div>
17  22      );
18  23    }
```

MODIFIÉ **src/pages/HomePage.jsx** +10 −1

```
        @@ -1,9 +1,13 @@
1   1   import { useQuery } from "@tanstack/react-query";
    2   + import { useDispatch, useSelector } from "react-redux";
2   3   import { fetchCarsByYear } from "../api/actions";
    4   + import { selectFavorites, toggleFavorite } from "../store/favoritesSlice";
3   5   import CarCard from "../components/CarCard";
4   6   import Loading from "../components/Loading";
5   7
6   8   export default function HomePage() {
    9   +   const dispatch = useDispatch();
    10  +   const favorites = useSelector(selectFavorites);
7   11      const { data, isLoading, error } = useQuery({
8   12        queryKey: ["cars", 2012],
9   13        queryFn: () => fetchCarsByYear(2012),
        @@ -27,7 +31,12 @@
27  31          <h1 className="page-title">Car Catalog</h1>
28  32          <div className="car-grid">
29  33            {cars.map((car) => (
30      -           <CarCard key={car.id} car={car} />
    34  +           <CarCard
    35  +             key={car.id}
    36  +             car={car}
    37  +             isFavorite={favorites.some((fav) => fav.id === car.id)}
    38  +             onToggleFavorite={(c) => dispatch(toggleFavorite(c))}
    39  +           />
31  40           ))}
32  41          </div>
33  42        </div>
```

AJOUTÉ **src/store/favoritesSlice.js** +28 −0

```
        @@ -1,0 +1,28 @@
    1   + import { createSlice } from "@reduxjs/toolkit";
    2   +
    3   + const initialState = {
    4   +   favorites: [],
    5   + };
    6   +
    7   + const favoritesSlice = createSlice({
    8   +   name: "favorites",
    9   +   initialState,
```

```
10 +   reducers: {
11 +     // Si la voiture est déjà en favori on la retire, sinon on l'ajoute
12 +     toggleFavorite: (state, action) => {
13 +       const carExists = state.favorites.some((car) => car.id === action.payload.id);
14 +       if (carExists) {
15 +         state.favorites = state.favorites.filter((car) => car.id !== action.payload.id);
16 +       } else {
17 +         state.favorites.push(action.payload);
18 +       }
19 +     },
20 +   },
21 + });
22 +
23 + export const { toggleFavorite } = favoritesSlice.actions;
24 +
25 + // Selectors
26 + export const selectFavorites = (state) => state.favorites.favorites;
27 +
28 + export default favoritesSlice.reducer;
```

AJOUTÉ **src/store/store.js** +8 −0

```
@@ -1,0 +1,8 @@
1 + import { configureStore } from "@reduxjs/toolkit";
2 + import favoritesReducer from "./favoritesSlice";
3 +
4 + export const store = configureStore({
5 +   reducer: {
6 +     favorites: favoritesReducer,
7 +   },
8 + });
```

Étape 4 → 5 etape-4-favoris → etape-5-login

L'authentification : storage (token + user), login() avec useMutation, interceptor axios, Header (user + logout)

MODIFIÉ src/api/actions.js +4 −0

```
@@ -1,4 +1,8 @@
  import axiosQuery from "../lib/axiosQuery";
+
+ // Login : l'API renvoie { session (le token), userDetails }
+ export const login = (userName, password) =>
+   axiosQuery("post", "/api/login", { userName, password, verified: true, blocked: false, delay: 0 });

  // Liste des voitures d'une année (la page Home)
  export const fetchCarsByYear = (year) => axiosQuery("get", `/api/cars/year/${year}`);
```

MODIFIÉ src/components/Header.jsx +34 −2

```
@@ -1,6 +1,29 @@
- import { Link } from "react-router-dom";
+ import { useEffect, useState } from "react";
+ import { Link, useNavigate } from "react-router-dom";
+ import { storage } from "../utils/storage";

  export default function Header() {
+   const navigate = useNavigate();
+   const [user, setUser] = useState(storage.getUser);
+
+   // On écoute les changements de l'utilisateur (login dans cet onglet ou dans un autre)
+   useEffect(() => {
+     const refreshUser = () => setUser(storage.getUser());
+     window.addEventListener("storage", refreshUser);
+     window.addEventListener("userUpdated", refreshUser);
+     return () => {
+       window.removeEventListener("storage", refreshUser);
+       window.removeEventListener("userUpdated", refreshUser);
+     };
+   }, []);
+
+   const handleLogout = () => {
+     storage.removeToken();
+     storage.removeUser();
+     setUser(null);
+     navigate("/login");
+   };
+
    return (
      <header className="header">
        <div className="header-content">
@@ -19,7 +42,16 @@
            </Link>
          </nav>
          <div className="user-section">
-           <span className="username">Guest</span>
+           <span className="username">{user?.userName || "Guest"}</span>
+           {user ? (
+             <button className="btn btn-sm" onClick={handleLogout}>
+               Logout
+             </button>
+           ) : (
+             <Link to="/login" className="btn btn-sm">
```

```
      52 +              Login
      53 +            </Link>
      54 +          )}
23    55          </div>
24    56        </div>
25    57      </header>
```

MODIFIÉ **src/lib/axiosQuery.js** +10 −0

```
@@ -1,5 +1,6 @@
1     1   import axios from "axios";
2     2   import { BASE_URL } from "../api/config";
      3 + import { storage } from "../utils/storage";
3     4
4     5   const axiosInstance = axios.create({
5     6     baseURL: BASE_URL,
@@ -10,6 +11,15 @@
10    11    timeout: 30000,
11    12  });
12    13
      14 + // Interceptor : on ajoute le token à chaque requête, s'il existe
      15 + axiosInstance.interceptors.request.use((config) => {
      16 +   const token = storage.getToken();
      17 +   if (token) {
      18 +     config.headers.Authorization = `Bearer ${token}`;
      19 +   }
      20 +   return config;
      21 + });
      22 +
13    23  // Un seul point d'entrée pour tous les appels API : on récupère directement les données
14    24  const axiosQuery = async (method, url, data) => {
15    25    const response = method === "post" ? await axiosInstance.post(url, data) : await axiosIns
tance.get(url);
```

MODIFIÉ **src/pages/LoginPage.jsx** +26 −3

```
@@ -1,11 +1,29 @@
      1 + import { useMutation } from "@tanstack/react-query";
1     2   import { useNavigate } from "react-router-dom";
      3 + import { login } from "../api/actions";
      4 + import { storage } from "../utils/storage";
2     5
3     6   export default function LoginPage() {
4     7     const navigate = useNavigate();
5     8
      9 +   const loginMutation = useMutation({
      10 +     mutationFn: ({ userName, password }) => login(userName, password),
      11 +     onSuccess: (response) => {
      12 +       // On garde le token et l'utilisateur dans le localStorage
      13 +       if (response.session) storage.setToken(response.session);
      14 +       if (response.userDetails) {
      15 +         storage.setUser(response.userDetails);
      16 +         // On prévient le Header que l'utilisateur a changé
      17 +         window.dispatchEvent(new CustomEvent("userUpdated"));
      18 +       }
      19 +       navigate("/");
      20 +     },
      21 +   });
      22 +
6     23    const handleSubmit = (e) => {
7     24      e.preventDefault();
8           -   navigate("/");
      25 +   const formData = new FormData(e.target);
```

```
      26  +     loginMutation.mutate({ userName: formData.get("userName"), password: formData.get("password") });
   9  27      };
  10  28
  11  29      return (
          @@ -14,6 +32,11 @@
  14  32            <h1 className="login-title">Welcome Back</h1>
  15  33            <p className="subtitle">Sign in to your account</p>
  16  34            <form onSubmit={handleSubmit} className="login-form">
      35  +           {loginMutation.isError && (
      36  +             <p className="error-text">
      37  +               {loginMutation.error?.response?.data?.message || loginMutation.error?.message || "Login failed. Please try again."}
      38  +             </p>
      39  +           )}
  17  40              <div className="form-group">
  18  41                <label htmlFor="userName" className="form-label">
  19  42                  Username
          @@ -26,8 +49,8 @@
  26  49                </label>
  27  50                <input type="password" id="password" name="password" className="form-input" placeholder="Enter your password" required />
  28  51              </div>
  29      -           <button type="submit" className="btn">
  30      -             Sign In
      52  +           <button type="submit" className="btn" disabled={loginMutation.isPending}>
      53  +             {loginMutation.isPending ? "Signing In..." : "Sign In"}
  31  54              </button>
  32  55            </form>
  33  56          </div>
```

AJOUTÉ **src/utils/storage.js** +16 −0

```
      @@ -1,0 +1,16 @@
    1 + // Le token et l'utilisateur sont gardés dans le localStorage (la session survit au rechargement)
    2 + const TOKEN_KEY = "auth_token";
    3 + const USER_KEY = "auth_user";
    4 +
    5 + export const storage = {
    6 +   setToken: (token) => localStorage.setItem(TOKEN_KEY, token),
    7 +   getToken: () => localStorage.getItem(TOKEN_KEY),
    8 +   removeToken: () => localStorage.removeItem(TOKEN_KEY),
    9 +
   10 +   setUser: (user) => localStorage.setItem(USER_KEY, JSON.stringify(user)),
   11 +   getUser: () => {
   12 +     const user = localStorage.getItem(USER_KEY);
   13 +     return user ? JSON.parse(user) : null;
   14 +   },
   15 +   removeUser: () => localStorage.removeItem(USER_KEY),
   16 + };
```

Étape 5 → 6 etape-5-login → etape-6-routes-protegees

Les loaders : requireAuth sur /favorites, redirectIfAuthenticated sur /login

MODIFIÉ `src/App.jsx` +3 −2

```
@@ -6,15 +6,16 @@
  import CarDetailPage from "./pages/CarDetailPage";
  import ApiDemoPage from "./pages/ApiDemoPage";
  import NotFoundPage from "./pages/NotFoundPage";
+ import { requireAuth, redirectIfAuthenticated } from "./utils/authLoader";

  const router = createBrowserRouter([
-   { path: "/login", element: <LoginPage /> },
+   { path: "/login", element: <LoginPage />, loader: redirectIfAuthenticated },
    {
      path: "/",
      element: <MainLayout />,
      children: [
        { index: true, element: <HomePage /> },
-       { path: "favorites", element: <FavoritesPage /> },
+       { path: "favorites", element: <FavoritesPage />, loader: requireAuth },
        { path: "car/:id", element: <CarDetailPage /> },
        { path: "api-demo", element: <ApiDemoPage /> },
        { path: "*", element: <NotFoundPage /> },
```

AJOUTÉ `src/utils/authLoader.js` +19 −0

```
@@ -1,0 +1,19 @@
+ import { redirect } from "react-router-dom";
+ import { storage } from "./storage";
+
+ // Loader de /favorites : pas de token => redirection vers /login (avant même d'afficher la page)
+ export const requireAuth = () => {
+   const token = storage.getToken();
+   if (!token) {
+     throw redirect("/login");
+   }
+   return { token };
+ };
+
+ // Loader de /login : déjà connecté => retour à l'accueil
+ export const redirectIfAuthenticated = () => {
+   if (storage.getToken()) {
+     throw redirect("/");
+   }
+   return null;
+ };
```

Étape 6 → 7 etape-6-routes-protegees → etape-7-error-boundary

errorElement : un écran d'erreur avec « Try again », et un bouton pour provoquer une erreur

MODIFIÉ `src/App.jsx` +2 −0

```
@@ -6,6 +6,7 @@
  import CarDetailPage from "./pages/CarDetailPage";
  import ApiDemoPage from "./pages/ApiDemoPage";
  import NotFoundPage from "./pages/NotFoundPage";
+ import ErrorBoundary from "./components/ErrorBoundary";
  import { requireAuth, redirectIfAuthenticated } from "./utils/authLoader";

  const router = createBrowserRouter([
@@ -13,6 +14,7 @@
    {
      path: "/",
      element: <MainLayout />,
+     errorElement: <ErrorBoundary />,
      children: [
        { index: true, element: <HomePage /> },
        { path: "favorites", element: <FavoritesPage />, loader: requireAuth },
```

AJOUTÉ `src/components/ErrorBoundary.jsx` +26 −0

```
@@ -1,0 +1,26 @@
+ import { useRouteError, useNavigate, Link } from "react-router-dom";
+ import { AlertIcon } from "./Icons";
+
+ // errorElement de React Router : une erreur dans une page affiche cet écran au lieu de cas
ser l'application
+ export default function ErrorBoundary() {
+   const error = useRouteError();
+   const navigate = useNavigate();
+
+   console.error("ErrorBoundary:", error);
+
+   return (
+     <div className="center-page error-content">
+       <AlertIcon size={64} />
+       <h1 className="error-title">Something went wrong!</h1>
+       <p className="muted">{error?.message || "An unexpected error occurred"}</p>
+       <div className="button-group">
+         <button className="btn" onClick={() => navigate(0)}>
+           Try again
+         </button>
+         <Link to="/" className="btn">
+           Go Home
+         </Link>
+       </div>
+     </div>
+   );
+ }
```

MODIFIÉ `src/pages/ApiDemoPage.jsx` +12 −0

```
@@ -5,6 +5,12 @@
  export default function ApiDemoPage() {
    const [loading, setLoading] = useState(false);
    const [result, setResult] = useState(null); // { ok: true|false, message }
+   const [crash, setCrash] = useState(false);
+
+   // Pour tester l'Error Boundary : une erreur pendant le render
```

```
          11  +   if (crash) {
          12  +     throw new Error("Crash volontaire pour tester l'Error Boundary");
          13  +   }
     8    14
     9    15      const simulate = (ok) => {
    10    16        setResult(null);
                @@ -31,6 +37,12 @@
    31    37              </button>
    32    38            </div>
    33    39
          40  +         <div className="button-group">
          41  +           <button onClick={() => setCrash(true)} className="btn">
          42  +             Crash the page (Error Boundary)
          43  +           </button>
          44  +         </div>
          45  +
    34    46            {loading && (
    35    47              <div className="message-box loading-box">
    36    48                <div className="spinner"></div>
```

MODIFIÉ **src/styles.css** +13 −0

```
                @@ -445,6 +445,19 @@
   445   445
   446   446    /* ===== Erreur et 404 ===== */
   447   447
          448  + .error-content svg {
          449  +   color: var(--error);
          450  + }
          451  +
          452  + .error-title {
          453  +   font-size: 1.5rem;
          454  +   font-weight: 700;
          455  + }
          456  +
          457  + .error-content .button-group {
          458  +   margin: 1rem 0 0;
          459  + }
          460  +
   448   461    .not-found-title {
   449   462      font-size: 6rem;
   450   463      font-weight: 700;
```

Étape 7 → 8 etape-7-error-boundary → etape-8-tests

Les tests : Vitest (le slice des favoris), Playwright (4 parcours e2e)

MODIFIÉ `package.json` +7 −2

```
@@ -6,7 +6,10 @@
    "scripts": {
      "dev": "vite",
      "build": "vite build",
-     "preview": "vite preview"
+     "preview": "vite preview",
+     "test": "vitest",
+     "test:e2e": "playwright test",
+     "test:e2e:ui": "playwright test --ui"
    },
    "dependencies": {
      "@reduxjs/toolkit": "^2.11.2",
@@ -19,7 +22,9 @@
      "react-router-dom": "^7.11.0"
    },
    "devDependencies": {
+     "@playwright/test": "^1.56.0",
      "@vitejs/plugin-react": "^5.1.1",
-     "vite": "^7.2.4"
+     "vite": "^7.2.4",
+     "vitest": "^3.2.4"
    }
  }
```

AJOUTÉ `playwright.config.js` +26 −0

```
@@ -1,0 +1,26 @@
+ import { defineConfig, devices } from '@playwright/test';
+
+ export default defineConfig({
+   testDir: './tests/e2e',
+   fullyParallel: true,
+   forbidOnly: !!process.env.CI,
+   retries: process.env.CI ? 2 : 0,
+   workers: process.env.CI ? 1 : undefined,
+   reporter: 'html',
+   use: {
+     baseURL: 'http://localhost:5173',
+     trace: 'on-first-retry',
+   },
+   projects: [
+     {
+       name: 'chromium',
+       use: { ...devices['Desktop Chrome'] },
+     },
+   ],
+   webServer: {
+     command: 'npm run dev',
+     url: 'http://localhost:5173',
+     reuseExistingServer: !process.env.CI,
+   },
+ });
+
```

AJOUTÉ `src/store/favoritesSlice.test.js` +62 −0

```
@@ -1,0 +1,62 @@
```

```
+ import { describe, it, expect } from 'vitest';
+ import favoritesReducer, { toggleFavorite } from './favoritesSlice';
+
+ describe('favoritesSlice', () => {
+   const mockCar = {
+     id: 1,
+     car: 'Tesla',
+     car_model: 'Model S',
+     car_color: 'Red',
+     car_model_year: 2024,
+     car_vin: 'TEST123',
+     price: '$79,990',
+     availability: true,
+   };
+
+   const mockCar2 = {
+     id: 2,
+     car: 'BMW',
+     car_model: 'i8',
+     car_color: 'Blue',
+     car_model_year: 2023,
+     car_vin: 'TEST456',
+     price: '$147,500',
+     availability: false,
+   };
+
+   it('should return initial state', () => {
+     expect(favoritesReducer(undefined, { type: 'unknown' })).toEqual({
+       favorites: [],
+     });
+   });
+
+   it('should add a car to favorites', () => {
+     const state = favoritesReducer(undefined, toggleFavorite(mockCar));
+     expect(state.favorites).toHaveLength(1);
+     expect(state.favorites[0]).toEqual(mockCar);
+   });
+
+   it('should remove a car from favorites when toggled again', () => {
+     const initialState = { favorites: [mockCar] };
+     const state = favoritesReducer(initialState, toggleFavorite(mockCar));
+     expect(state.favorites).toHaveLength(0);
+   });
+
+   it('should add multiple cars to favorites', () => {
+     let state = favoritesReducer(undefined, toggleFavorite(mockCar));
+     state = favoritesReducer(state, toggleFavorite(mockCar2));
+
+     expect(state.favorites).toHaveLength(2);
+     expect(state.favorites).toContainEqual(mockCar);
+     expect(state.favorites).toContainEqual(mockCar2);
+   });
+
+   it('should remove specific car without affecting others', () => {
+     const initialState = { favorites: [mockCar, mockCar2] };
+     const state = favoritesReducer(initialState, toggleFavorite(mockCar));
+
+     expect(state.favorites).toHaveLength(1);
+     expect(state.favorites[0]).toEqual(mockCar2);
+   });
+ });
+
```

AJOUTÉ **tests/e2e/favorites.spec.js** +88 −0

```
@@ -1,0 +1,88 @@
+ import { test, expect } from '@playwright/test';
+
+ test.describe('Favorites Feature', () => {
+   test.beforeEach(async ({ page }) => {
+     // Set up authentication in localStorage
+     await page.goto('/');
+     await page.evaluate(() => {
+       localStorage.setItem('auth_token', 'test-token-123');
+       localStorage.setItem('auth_user', JSON.stringify({
+         userName: 'Test User',
+         password: 'test',
+         verified: true,
+         blocked: false,
+       }));
+     });
+
+     // Visit the home page
+     await page.goto('/');
+
+     // Wait for cars to load - use heading role to be more specific
+     await expect(page.getByRole('heading', { name: 'Car Catalog' })).toBeVisible();
+     await expect(page.locator('.car-card').first()).toBeVisible({ timeout: 10000 });
+   });
+
+   test('should add a car to favorites when clicking favorite button', async ({ page }) => {
+     // Click the first favorite button
+     const firstCard = page.locator('.car-card').first();
+     await firstCard.locator('.favorite-btn').click();
+
+     // Check if the button has the active class
+     await expect(firstCard.locator('.favorite-btn')).toHaveClass(/favorite-active/);
+
+     // Navigate to favorites page
+     await page.getByRole('link', { name: 'Favorites' }).click();
+
+     // Verify the car is in favorites
+     await expect(page).toHaveURL(/.*favorites/);
+     await expect(page.locator('.car-card').first()).toBeVisible();
+   });
+
+   test('should remove a car from favorites when clicking favorite button again', async ({ p
age }) => {
+     // Add car to favorites from home page
+     const firstCard = page.locator('.car-card').first();
+     await firstCard.locator('.favorite-btn').click();
+     await expect(firstCard.locator('.favorite-btn')).toHaveClass(/favorite-active/);
+
+     // Navigate to favorites
+     await page.getByRole('link', { name: 'Favorites' }).click();
+     await expect(page.locator('.car-card').first()).toBeVisible();
+
+     // Remove from favorites
+     await page.locator('.car-card').first().locator('.favorite-btn').click();
+
+     // Should show empty state
+     await expect(page.getByText('No favorites yet')).toBeVisible();
+   });
+
+   test('should persist favorites when navigating between pages', async ({ page }) => {
+     // Add car to favorites
+     const firstCard = page.locator('.car-card').first();
+     await firstCard.locator('.favorite-btn').click();
+
+     // Navigate to favorites
```

```
+     await page.getByRole('link', { name: 'Favorites' }).click();
+     await expect(page.locator('.car-card').first()).toBeVisible();
+
+     // Go back to home
+     await page.getByRole('link', { name: 'Home' }).click();
+
+     // Favorite button should still be active
+     await expect(page.locator('.car-card').first().locator('.favorite-btn')).toHaveClass(/favorite-active/);
+   });
+
+   test('should display multiple favorites', async ({ page }) => {
+     // Add first car to favorites
+     await page.locator('.car-card').first().locator('.favorite-btn').click();
+
+     // Add second car to favorites
+     await page.locator('.car-card').nth(1).locator('.favorite-btn').click();
+
+     // Navigate to favorites
+     await page.getByRole('link', { name: 'Favorites' }).click();
+
+     // Should have 2 cars
+     await expect(page.locator('.car-card')).toHaveCount(2);
+   });
+ });
+
```

MODIFIÉ **vite.config.js** +4 −0

```
@@ -12,4 +12,8 @@
        },
      },
    },
+   // Vitest : les tests unitaires (*.test.js) ; les tests e2e (Playwright) sont dans tests/e2e
+   test: {
+     include: ["src/**/*.test.js"],
+   },
  });
```

Étape 8 → 9 etape-8-tests → etape-9-architecture

L'architecture feature-based : atomic design (components/), un écran = View / Scene / Adapter, hooks par feature, tests des adapters (voir ARCHITECTURE.md)

AJOUTÉ **ARCHITECTURE.md** +66 −0

```
@@ -1,0 +1,66 @@
+ # Étape 9 : architecture
+
+ Même application que l'étape 8 (mêmes routes, même rendu, mêmes tests e2e). Seule l'organisation de `src/` change.
+
+ ## Les dossiers
+
+ | Dossier | Rôle |
+ |---|---|
+ | `app/` | Le démarrage de l'application : `router.jsx` (routes + loaders), `providers.jsx` (Redux + React Query + devtools), `store.js`, `RootLayout.jsx` (relie l'utilisateur connecté au template). |
+ | `lib/` | Les outils techniques partagés : `axiosQuery.js` (instance axios + token), `queryClient.js`, `config.js`. |
+ | `components/` | Les composants UI réutilisables, en **atomic design**. Ils ne reçoivent que des props simples (texte, nombre, booléen, fonction) et ne connaissent ni l'API, ni Redux, ni le localStorage. |
+ | `components/atoms` | Les plus petites briques : `Button`/`ButtonLink`, icônes, `ColorDot`, `StatusBadge`, `Price`, `Spinner`. |
+ | `components/molecules` | Quelques atoms assemblés : `FavoriteButton`, `InfoRow`, `DetailItem`, `FormField`, `Loading`, `ErrorMessage`, `StatusMessage`. |
+ | `components/organisms` | Des blocs complets : `CarCard`, `CarGrid`, `LoginForm`. |
+ | `components/templates` | La mise en page : `MainLayout`, `Header`, `Footer`. |
+ | `features/` | Le code **métier**, rangé par fonctionnalité : `cars`, `favorites`, `auth`, `errors`, `api-demo`. |
+
+ Dans une feature :
+
+ - `api/` : les appels HTTP (`carsApi.js`, `authApi.js`) ;
+ - `hooks/` : les hooks React Query / Redux (`useCars`, `useCarDetails`, `useFavorites`, `useCurrentUser`, `useLogin`) ; les clés React Query sont dans `queryKeys.js` ;
+ - `utils/`, `adapters/` : des fonctions pures (`colorUtils`, `carAdapter`, `authLoader`) ;
+ - `screens/<Ecran>/` : un écran = 4 fichiers.
+
+ ## Un écran = View + adapter + Scene
+
+ | Fichier | Rôle |
+ |---|---|
+ | `<Ecran>.view.jsx` | Le **conteneur** : appelle les hooks, gère chargement / erreur / vide, appelle l'adapter. |
+ | `<Ecran>.adapter.js` | Une fonction **pure** : données de l'API (DTO) vers modèle prêt à afficher. |
+ | `<Ecran>.scene.jsx` | La **présentation** : ne reçoit que des props et assemble les composants de `components/`. |
+ | `index.js` | Réexporte l'écran. Le router importe toujours par ce fichier. |
+
+ Un écran sans données (la 404) n'a qu'une Scene : on n'ajoute pas de fichier vide.
+
+ ### Exemple : la page d'accueil
+
+ ```
+ HomeScreen.view   ->  useCars(2012)           (hook React Query)
+                   ->  fetchCarsByYear(2012)   (api, via axiosQuery)
+                   <-  { Cars: [{ id: 12, car: "Audi", car_model: "Q7", car_color: "Red", price: "$23,456.78", availability: true, ... }] }
```

```
+                  ->  adaptHomeCars(data, favoriteIds)   (adapter : 6 voitures maximum)
+                  <-  [{ id: 12, title: "Audi Q7", colorHex: "#e74c3c", priceLabel: "$23,456.78",
+                          isAvailable: true, statusLabel: "Available", detailsUrl: "/car/12", isFavorite: false }]
+                  ->  <HomeScreenScene cars={...} onToggleFavorite={...} />
+                        -> <CarGrid> -> <CarCard title="Audi Q7" colorHex="#e74c3c" ... />
+ ```
+
+ L'adapter gère aussi les valeurs manquantes : une couleur inconnue donne du gris, un prix absent affiche « — », une réponse vide donne une liste vide.
+
+ Les favoris stockés dans Redux restent les **objets bruts de l'API**. La View retrouve l'objet brut à partir de l'id cliqué (`findCarById`) avant d'appeler `toggleFavorite`.
+
+ ## Règles d'import
+
+ - On utilise l'alias `@` (vers `src/`) pour aller d'un dossier à un autre : `@/components/molecules`, `@/features/favorites/hooks`.
+ - On passe toujours par les `index.js` qui existent (`components/atoms`, `.../hooks`, `.../screens/<Ecran>`).
+ - `components/` n'importe jamais `features/`. C'est `app/RootLayout.jsx` qui donne au `Header` le nom de l'utilisateur et la fonction de déconnexion.
+
+ ## Les tests
+
+ | Type | Outil | Ce qu'on teste |
+ |---|---|---|
+ | Unitaire | Vitest (`npm test`, environnement node) | Les fonctions pures : les adapters (cas normaux, valeurs manquantes, limite de 6), `colorUtils`, `authLoader` (avec un stockage simulé), le slice des favoris. Ces tests sont rapides et n'ont besoin ni de navigateur ni de réseau. |
+ | End-to-end | Playwright (`npm run test:e2e`) | Le parcours réel dans le navigateur, avec la vraie API : ajouter, retirer et retrouver des favoris. |
+
+ Grâce à la séparation View / adapter / Scene, la logique de transformation se teste sans afficher de composant. Les tests e2e vérifient que tout fonctionne ensemble.
```

SUPPRIMÉ **src/App.jsx** +0 −30

```
@@ -1,30 +1,0 @@
- import { createBrowserRouter, RouterProvider } from "react-router-dom";
- import MainLayout from "./layouts/MainLayout";
- import HomePage from "./pages/HomePage";
- import LoginPage from "./pages/LoginPage";
- import FavoritesPage from "./pages/FavoritesPage";
- import CarDetailPage from "./pages/CarDetailPage";
- import ApiDemoPage from "./pages/ApiDemoPage";
- import NotFoundPage from "./pages/NotFoundPage";
- import ErrorBoundary from "./components/ErrorBoundary";
- import { requireAuth, redirectIfAuthenticated } from "./utils/authLoader";
-
- const router = createBrowserRouter([
-   { path: "/login", element: <LoginPage />, loader: redirectIfAuthenticated },
-   {
-     path: "/",
-     element: <MainLayout />,
-     errorElement: <ErrorBoundary />,
-     children: [
-       { index: true, element: <HomePage /> },
-       { path: "favorites", element: <FavoritesPage />, loader: requireAuth },
-       { path: "car/:id", element: <CarDetailPage /> },
-       { path: "api-demo", element: <ApiDemoPage /> },
-       { path: "*", element: <NotFoundPage /> },
-     ],
```

```
25  -   },
26  - ]);
27  -
28  - export default function App() {
29  -   return <RouterProvider router={router} />;
30  - }
```

SUPPRIMÉ **src/api/actions.js** +0 −11

```
@@ -1,11 +1,0 @@
1   - import axiosQuery from "../lib/axiosQuery";
2   -
3   - // Login : l'API renvoie { session (le token), userDetails }
4   - export const login = (userName, password) =>
5   -   axiosQuery("post", "/api/login", { userName, password, verified: true, blocked: false, delay: 0 });
6   -
7   - // Liste des voitures d'une année (la page Home)
8   - export const fetchCarsByYear = (year) => axiosQuery("get", `/api/cars/year/${year}`);
9   -
10  - // Détails d'une voiture (la page /car/:id)
11  - export const fetchCarById = (id) => axiosQuery("get", `/api/cars/${id}`);
```

SUPPRIMÉ **src/api/config.js** +0 −2

```
@@ -1,2 +1,0 @@
1   - // Les requêtes passent par le proxy Vite (/api) : pas besoin d'URL complète en dev
2   - export const BASE_URL = "";
```

AJOUTÉ **src/app/App.jsx** +6 −0

```
@@ -1,0 +1,6 @@
1   + import { RouterProvider } from "react-router-dom";
2   + import { router } from "./router";
3   +
4   + export default function App() {
5   +   return <RouterProvider router={router} />;
6   + }
```

AJOUTÉ **src/app/RootLayout.jsx** +14 −0

```
@@ -1,0 +1,14 @@
1   + import { Outlet } from "react-router-dom";
2   + import { MainLayout } from "@/components/templates";
3   + import { useCurrentUser } from "@/features/auth/hooks";
4   +
5   + // Fait le lien entre la feature auth (l'utilisateur connecté) et le template, qui ne reçoit que des props
6   + export default function RootLayout() {
7   +   const { user, logout } = useCurrentUser();
8   +
9   +   return (
10  +     <MainLayout userName={user?.userName || "Guest"} isLoggedIn={Boolean(user)} onLogout={logout}>
11  +       <Outlet />
12  +     </MainLayout>
13  +   );
14  + }
```

AJOUTÉ **src/app/providers.jsx** +17 −0

```
@@ -1,0 +1,17 @@
1   + import { Provider } from "react-redux";
2   + import { QueryClientProvider } from "@tanstack/react-query";
3   + import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
```

```
+ import { queryClient } from "@/lib/queryClient";
+ import { store } from "./store";
+
+ // Tous les providers globaux au même endroit : Redux (favoris) + React Query (données API)
+ export default function AppProviders({ children }) {
+   return (
+     <Provider store={store}>
+       <QueryClientProvider client={queryClient}>
+         {children}
+         <ReactQueryDevtools initialIsOpen={false} />
+       </QueryClientProvider>
+     </Provider>
+   );
+ }
```

AJOUTÉ **src/app/router.jsx** +26 −0

```
@@ -1,0 +1,26 @@
+ import { createBrowserRouter } from "react-router-dom";
+ import RootLayout from "./RootLayout";
+ import { HomeScreen } from "@/features/cars/screens/HomeScreen";
+ import { CarDetailsScreen } from "@/features/cars/screens/CarDetailsScreen";
+ import { FavoritesScreen } from "@/features/favorites/screens/FavoritesScreen";
+ import { LoginScreen } from "@/features/auth/screens/LoginScreen";
+ import { ApiDemoScreen } from "@/features/api-demo/screens/ApiDemoScreen";
+ import { ErrorScreen } from "@/features/errors/screens/ErrorScreen";
+ import { NotFoundScreen } from "@/features/errors/screens/NotFoundScreen";
+ import { requireAuth, redirectIfAuthenticated } from "@/features/auth/utils/authLoader";
+
+ export const router = createBrowserRouter([
+   { path: "/login", element: <LoginScreen />, loader: redirectIfAuthenticated },
+   {
+     path: "/",
+     element: <RootLayout />,
+     errorElement: <ErrorScreen />,
+     children: [
+       { index: true, element: <HomeScreen /> },
+       { path: "favorites", element: <FavoritesScreen />, loader: requireAuth },
+       { path: "car/:id", element: <CarDetailsScreen /> },
+       { path: "api-demo", element: <ApiDemoScreen /> },
+       { path: "*", element: <NotFoundScreen /> },
+     ],
+   },
+ ]);
```

AJOUTÉ **src/app/store.js** +8 −0

```
@@ -1,0 +1,8 @@
+ import { configureStore } from "@reduxjs/toolkit";
+ import favoritesReducer from "@/features/favorites/favoritesSlice";
+
+ export const store = configureStore({
+   reducer: {
+     favorites: favoritesReducer,
+   },
+ });
```

SUPPRIMÉ **src/components/CarCard.jsx** +0 −42

```
@@ -1,42 +1,0 @@
- import { Link } from "react-router-dom";
- import { HeartIcon } from "./Icons";
- import { getColorHex } from "../utils/colorUtils";
-
- // La carte d'une voiture : utilisée par la page Home et la page Favoris
```

```
- export default function CarCard({ car, isFavorite = false, onToggleFavorite }) {
-   return (
-     <div className="card car-card">
-       <div className="card-header">
-         <h2 className="car-name">
-           {car.car} {car.car_model}
-         </h2>
-         <button className={`favorite-btn ${isFavorite ? "favorite-active" : ""}`} onClick={() => onToggleFavorite?.(car)}>
-           <HeartIcon filled={isFavorite} />
-         </button>
-       </div>
-       <div className="car-info">
-         <div className="info-row">
-           <span className="label">Color:</span>
-           <div className="color-indicator" style={{ backgroundColor: getColorHex(car.car_color) }}></div>
-         </div>
-         <div className="info-row">
-           <span className="label">Status:</span>
-           <span className={car.availability ? "status-available" : "status-unavailable"}>
-             {car.availability ? "Available" : "Unavailable"}
-           </span>
-         </div>
-         <div className="info-row">
-           <span className="label">Price:</span>
-           <span className="price">{car.price}</span>
-         </div>
-         <div className="info-row">
-           <span className="label">Year:</span>
-           <span>{car.car_model_year}</span>
-         </div>
-       </div>
-       <Link to={`/car/${car.id}`} className="btn">
-         View Details
-       </Link>
-     </div>
-   );
- }
```

SUPPRIMÉ **src/components/ErrorBoundary.jsx** +0 −26

```
@@ -1,26 +1,0 @@
- import { useRouteError, useNavigate, Link } from "react-router-dom";
- import { AlertIcon } from "./Icons";
-
- // errorElement de React Router : une erreur dans une page affiche cet écran au lieu de casser l'application
- export default function ErrorBoundary() {
-   const error = useRouteError();
-   const navigate = useNavigate();
-
-   console.error("ErrorBoundary:", error);
-
-   return (
-     <div className="center-page error-content">
-       <AlertIcon size={64} />
-       <h1 className="error-title">Something went wrong!</h1>
-       <p className="muted">{error?.message || "An unexpected error occurred"}</p>
-       <div className="button-group">
-         <button className="btn" onClick={() => navigate(0)}>
-           Try again
-         </button>
-         <Link to="/" className="btn">
-           Go Home
```

```
22 -         </Link>
23 -       </div>
24 -     </div>
25 -   );
26 - }
```

SUPPRIMÉ **src/components/Footer.jsx** +0 −7

```
@@ -1,7 +1,0 @@
1 - export default function Footer() {
2 -   return (
3 -     <footer className="footer">
4 -       <p>&copy; 2026 Car Catalog. All rights reserved.</p>
5 -     </footer>
6 -   );
7 - }
```

SUPPRIMÉ **src/components/Header.jsx** +0 −59

```
@@ -1,59 +1,0 @@
1 - import { useEffect, useState } from "react";
2 - import { Link, useNavigate } from "react-router-dom";
3 - import { storage } from "../utils/storage";
4 -
5 - export default function Header() {
6 -   const navigate = useNavigate();
7 -   const [user, setUser] = useState(storage.getUser);
8 -
9 -   // On écoute les changements de l'utilisateur (login dans cet onglet ou dans un autre)
10 -   useEffect(() => {
11 -     const refreshUser = () => setUser(storage.getUser());
12 -     window.addEventListener("storage", refreshUser);
13 -     window.addEventListener("userUpdated", refreshUser);
14 -     return () => {
15 -       window.removeEventListener("storage", refreshUser);
16 -       window.removeEventListener("userUpdated", refreshUser);
17 -     };
18 -   }, []);
19 -
20 -   const handleLogout = () => {
21 -     storage.removeToken();
22 -     storage.removeUser();
23 -     setUser(null);
24 -     navigate("/login");
25 -   };
26 -
27 -   return (
28 -     <header className="header">
29 -       <div className="header-content">
30 -         <Link to="/" className="logo">
31 -           Car Catalog
32 -         </Link>
33 -         <nav className="nav">
34 -           <Link to="/" className="nav-link">
35 -             Home
36 -           </Link>
37 -           <Link to="/favorites" className="nav-link">
38 -             Favorites
39 -           </Link>
40 -           <Link to="/api-demo" className="nav-link">
41 -             API Demo
42 -           </Link>
43 -         </nav>
44 -         <div className="user-section">
45 -           <span className="username">{user?.userName || "Guest"}</span>
```

```
46  -           {user ? (
47  -             <button className="btn btn-sm" onClick={handleLogout}>
48  -               Logout
49  -             </button>
50  -           ) : (
51  -             <Link to="/login" className="btn btn-sm">
52  -               Login
53  -             </Link>
54  -           )}
55  -         </div>
56  -       </div>
57  -     </header>
58  -   );
59  - }
```

SUPPRIMÉ **src/components/Icons.jsx** +0 −36

```
@@ -1,36 +1,0 @@
1   - // Les icônes SVG, définies une seule fois et réutilisées partout
2   -
3   - export function HeartIcon({ filled = false }) {
4   -   return (
5   -     <svg width="24" height="24" viewBox="0 0 24 24" fill={filled ? "currentColor" : "none"} stroke="currentColor" strokeWidth="2">
6   -       <path d="M20.84 4.61a5.5 5.5 0 0 0-7.78 0L12 5.67l-1.06-1.06a5.5 5.5 0 0 0-7.78 7.78l1.06 1.06L12 21.23l7.78-7.78 1.06-1.06a5.5 5.5 0 0 0 0-7.78z" />
7   -     </svg>
8   -   );
9   - }
10  -
11  - export function AlertIcon({ size = 24 }) {
12  -   return (
13  -     <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
14  -       <circle cx="12" cy="12" r="10" />
15  -       <line x1="12" y1="8" x2="12" y2="12" />
16  -       <line x1="12" y1="16" x2="12.01" y2="16" />
17  -     </svg>
18  -   );
19  - }
20  -
21  - export function CheckIcon() {
22  -   return (
23  -     <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
24  -       <path d="M22 11.08V12a10 10 0 1 1-5.93-9.14" />
25  -       <polyline points="22 4 12 14.01 9 11.01" />
26  -     </svg>
27  -   );
28  - }
29  -
30  - export function BackIcon() {
31  -   return (
32  -     <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
33  -       <path d="M19 12H5M12 19l-7-7 7-7" />
34  -     </svg>
35  -   );
36  - }
```

SUPPRIMÉ **src/components/Loading.jsx** +0 −9

```
@@ -1,9 +1,0 @@
1   - // Le chargement : un spinner et un texte, réutilisé par toutes les pages
2   - export default function Loading({ text = "Loading..." }) {
```

```
-   return (
-     <div className="center-page">
-       <div className="spinner"></div>
-       <p>{text}</p>
-     </div>
-   );
- }
```

AJOUTÉ `src/components/atoms/Button.jsx` +22 −0

```
@@ -1,0 +1,22 @@
+ import { Link } from "react-router-dom";
+
+ // Classes CSS d'un bouton : "btn", puis "btn-sm" et/ou "btn-success" / "btn-danger"
+ const buttonClassName = (size, variant) => ["btn", size && `btn-${size}`, variant && `btn-${variant}`].filter(Boolean).join(" ");
+
+ // Un vrai <button> (action)
+ export function Button({ size, variant, type = "button", children, ...props }) {
+   return (
+     <button type={type} className={buttonClassName(size, variant)} {...props}>
+       {children}
+     </button>
+   );
+ }
+
+ // Un lien qui a l'apparence d'un bouton (navigation)
+ export function ButtonLink({ to, size, variant, children }) {
+   return (
+     <Link to={to} className={buttonClassName(size, variant)}>
+       {children}
+     </Link>
+   );
+ }
```

AJOUTÉ `src/components/atoms/ColorDot.jsx` +4 −0

```
@@ -1,0 +1,4 @@
+ // La pastille de couleur : reçoit directement un code couleur (#rrggbb)
+ export function ColorDot({ color }) {
+   return <div className="color-indicator" style={{ backgroundColor: color }}></div>;
+ }
```

AJOUTÉ `src/components/atoms/Icons.jsx` +36 −0

```
@@ -1,0 +1,36 @@
+ // Les icônes SVG, définies une seule fois et réutilisées partout
+
+ export function HeartIcon({ filled = false }) {
+   return (
+     <svg width="24" height="24" viewBox="0 0 24 24" fill={filled ? "currentColor" : "none"} stroke="currentColor" strokeWidth="2">
+       <path d="M20.84 4.61a5.5 5.5 0 0 0-7.78 0L12 5.67l-1.06-1.06a5.5 5.5 0 0 0-7.78 7.78l1.06 1.06L12 21.23l7.78-7.78 1.06-1.06a5.5 5.5 0 0 0 0-7.78z" />
+     </svg>
+   );
+ }
+
+ export function AlertIcon({ size = 24 }) {
+   return (
+     <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
+       <circle cx="12" cy="12" r="10" />
+       <line x1="12" y1="8" x2="12" y2="12" />
+       <line x1="12" y1="16" x2="12.01" y2="16" />
```

```
+     </svg>
+   );
+ }
+
+ export function CheckIcon() {
+   return (
+     <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
+       <path d="M22 11.08V12a10 10 0 1 1-5.93-9.14" />
+       <polyline points="22 4 12 14.01 9 11.01" />
+     </svg>
+   );
+ }
+
+ export function BackIcon() {
+   return (
+     <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
+       <path d="M19 12H5M12 19l-7-7 7-7" />
+     </svg>
+   );
+ }
```

AJOUTÉ **src/components/atoms/Price.jsx** +4 −0

```
@@ -1,0 +1,4 @@
+ // Un prix déjà formaté (le formatage est fait par l'adapter)
+ export function Price({ value, className }) {
+   return <span className={className ? `${className} price` : "price"}>{value}</span>;
+ }
```

AJOUTÉ **src/components/atoms/Spinner.jsx** +3 −0

```
@@ -1,0 +1,3 @@
+ export function Spinner() {
+   return <div className="spinner"></div>;
+ }
```

AJOUTÉ **src/components/atoms/StatusBadge.jsx** +5 −0

```
@@ -1,0 +1,5 @@
+ // Disponible (vert) ou indisponible (rouge) ; className permet d'ajouter une classe (ex. "detail-value")
+ export function StatusBadge({ isAvailable, label, className }) {
+   const statusClass = isAvailable ? "status-available" : "status-unavailable";
+   return <span className={className ? `${className} ${statusClass}` : statusClass}>{label}</span>;
+ }
```

AJOUTÉ **src/components/atoms/index.js** +6 −0

```
@@ -1,0 +1,6 @@
+ export { HeartIcon, AlertIcon, CheckIcon, BackIcon } from "./Icons";
+ export { Button, ButtonLink } from "./Button";
+ export { ColorDot } from "./ColorDot";
+ export { StatusBadge } from "./StatusBadge";
+ export { Price } from "./Price";
+ export { Spinner } from "./Spinner";
```

AJOUTÉ **src/components/molecules/DetailItem.jsx** +9 −0

```
@@ -1,0 +1,9 @@
+ // Une case de la grille de la page détails : petit label au-dessus, valeur en dessous
+ export function DetailItem({ label, children }) {
+   return (
```

```
+     <div className="detail-item">
+       <span className="detail-label">{label}</span>
+       {children}
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/molecules/ErrorMessage.jsx** +8 −0

```
@@ -1,0 +1,8 @@
+ // Une erreur pleine page (ex. l'API ne répond pas)
+ export function ErrorMessage({ message }) {
+   return (
+     <div className="center-page">
+       <p className="error-text">{message}</p>
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/molecules/FavoriteButton.jsx** +10 −0

```
@@ -1,0 +1,10 @@
+ import { HeartIcon } from "../atoms";
+
+ // Le bouton cœur : plein et rouge quand la voiture est en favori
+ export function FavoriteButton({ isActive, onToggle }) {
+   return (
+     <button className={`favorite-btn ${isActive ? "favorite-active" : ""}`} onClick={onToggle}>
+       <HeartIcon filled={isActive} />
+     </button>
+   );
+ }
```

AJOUTÉ **src/components/molecules/FormField.jsx** +11 −0

```
@@ -1,0 +1,11 @@
+ // Un champ de formulaire avec son label
+ export function FormField({ id, label, type = "text", placeholder, required = false }) {
+   return (
+     <div className="form-group">
+       <label htmlFor={id} className="form-label">
+         {label}
+       </label>
+       <input type={type} id={id} name={id} className="form-input" placeholder={placeholder} required={required} />
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/molecules/InfoRow.jsx** +9 −0

```
@@ -1,0 +1,9 @@
+ // Une ligne "Label: valeur" de la carte voiture
+ export function InfoRow({ label, children }) {
+   return (
+     <div className="info-row">
+       <span className="label">{`${label}:`}</span>
+       {children}
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/molecules/Loading.jsx** +11 −0

```
@@ -1,0 +1,11 @@
+ import { Spinner } from "../atoms";
+
+ // Le chargement pleine page : un spinner et un texte
+ export function Loading({ text = "Loading..." }) {
+   return (
+     <div className="center-page">
+       <Spinner />
+       <p>{text}</p>
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/molecules/StatusMessage.jsx** +17 −0

```
@@ -1,0 +1,17 @@
+ import { AlertIcon, CheckIcon, Spinner } from "../atoms";
+
+ const ICONS = {
+   loading: <Spinner />,
+   success: <CheckIcon />,
+   error: <AlertIcon />,
+ };
+
+ // Un encadré de message : variant = "loading" | "success" | "error"
+ export function StatusMessage({ variant, children }) {
+   return (
+     <div className={`message-box ${variant}-box`}>
+       {ICONS[variant]}
+       <span>{children}</span>
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/molecules/index.js** +7 −0

```
@@ -1,0 +1,7 @@
+ export { FavoriteButton } from "./FavoriteButton";
+ export { InfoRow } from "./InfoRow";
+ export { DetailItem } from "./DetailItem";
+ export { FormField } from "./FormField";
+ export { Loading } from "./Loading";
+ export { ErrorMessage } from "./ErrorMessage";
+ export { StatusMessage } from "./StatusMessage";
```

AJOUTÉ **src/components/organisms/CarCard.jsx** +29 −0

```
@@ -1,0 +1,29 @@
+ import { ButtonLink, ColorDot, Price, StatusBadge } from "../atoms";
+ import { FavoriteButton, InfoRow } from "../molecules";
+
+ // La carte d'une voiture : ne reçoit que des valeurs déjà prêtes à afficher (préparées par l'adapter)
+ export function CarCard({ title, colorHex, isAvailable, statusLabel, priceLabel, year, detailsUrl, isFavorite = false, onToggleFavorite }) {
+   return (
+     <div className="card car-card">
+       <div className="card-header">
+         <h2 className="car-name">{title}</h2>
+         <FavoriteButton isActive={isFavorite} onToggle={onToggleFavorite} />
+       </div>
+       <div className="car-info">
+         <InfoRow label="Color">
+           <ColorDot color={colorHex} />
```

```
+         </InfoRow>
+         <InfoRow label="Status">
+           <StatusBadge isAvailable={isAvailable} label={statusLabel} />
+         </InfoRow>
+         <InfoRow label="Price">
+           <Price value={priceLabel} />
+         </InfoRow>
+         <InfoRow label="Year">
+           <span>{year}</span>
+         </InfoRow>
+       </div>
+       <ButtonLink to={detailsUrl}>View Details</ButtonLink>
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/organisms/CarGrid.jsx** +12 −0

```
@@ -1,0 +1,12 @@
+ import { CarCard } from "./CarCard";
+
+ // La grille de cartes ; onToggleFavorite reçoit l'id de la voiture cliquée
+ export function CarGrid({ cars, onToggleFavorite }) {
+   return (
+     <div className="car-grid">
+       {cars.map((car) => (
+         <CarCard key={car.id} {...car} onToggleFavorite={() => onToggleFavorite(car.id)} />
+       ))}
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/organisms/LoginForm.jsx** +22 −0

```
@@ -1,0 +1,22 @@
+ import { Button } from "../atoms";
+ import { FormField } from "../molecules";
+
+ // Le formulaire de connexion : il lit les champs et renvoie { userName, password } au parent
+ export function LoginForm({ onSubmit, isPending, errorMessage }) {
+   const handleSubmit = (e) => {
+     e.preventDefault();
+     const formData = new FormData(e.target);
+     onSubmit({ userName: formData.get("userName"), password: formData.get("password") });
+   };
+
+   return (
+     <form onSubmit={handleSubmit} className="login-form">
+       {errorMessage && <p className="error-text">{errorMessage}</p>}
+       <FormField id="userName" label="Username" placeholder="Enter your username" required />
+       <FormField id="password" label="Password" type="password" placeholder="Enter your password" required />
+       <Button type="submit" disabled={isPending}>
+         {isPending ? "Signing In..." : "Sign In"}
+       </Button>
+     </form>
+   );
+ }
```

AJOUTÉ **src/components/organisms/index.js** +3 −0

```
@@ -1,0 +1,3 @@
+ export { CarCard } from "./CarCard";
```

```
2  + export { CarGrid } from "./CarGrid";
3  + export { LoginForm } from "./LoginForm";
```

AJOUTÉ **src/components/templates/Footer.jsx** +7 −0

```
@@ -1,0 +1,7 @@
1  + export function Footer() {
2  +   return (
3  +     <footer className="footer">
4  +       <p>&copy; 2026 Car Catalog. All rights reserved.</p>
5  +     </footer>
6  +   );
7  + }
```

AJOUTÉ **src/components/templates/Header.jsx** +38 −0

```
@@ -1,0 +1,38 @@
1  + import { Link } from "react-router-dom";
2  + import { Button, ButtonLink } from "../atoms";
3  +
4  + // L'en-tête : il ne connaît pas le stockage, il reçoit le nom affiché et l'action de décon
   nexion
5  + export function Header({ userName, isLoggedIn, onLogout }) {
6  +   return (
7  +     <header className="header">
8  +       <div className="header-content">
9  +         <Link to="/" className="logo">
10 +           Car Catalog
11 +         </Link>
12 +         <nav className="nav">
13 +           <Link to="/" className="nav-link">
14 +             Home
15 +           </Link>
16 +           <Link to="/favorites" className="nav-link">
17 +             Favorites
18 +           </Link>
19 +           <Link to="/api-demo" className="nav-link">
20 +             API Demo
21 +           </Link>
22 +         </nav>
23 +         <div className="user-section">
24 +           <span className="username">{userName}</span>
25 +           {isLoggedIn ? (
26 +             <Button size="sm" onClick={onLogout}>
27 +               Logout
28 +             </Button>
29 +           ) : (
30 +             <ButtonLink to="/login" size="sm">
31 +               Login
32 +             </ButtonLink>
33 +           )}
34 +         </div>
35 +       </div>
36 +     </header>
37 +   );
38 + }
```

AJOUTÉ **src/components/templates/MainLayout.jsx** +13 −0

```
@@ -1,0 +1,13 @@
1  + import { Header } from "./Header";
2  + import { Footer } from "./Footer";
3  +
4  + // La mise en page commune : en-tête, contenu de la page (children), pied de page
5  + export function MainLayout({ userName, isLoggedIn, onLogout, children }) {
```

```
+   return (
+     <div className="app-layout">
+       <Header userName={userName} isLoggedIn={isLoggedIn} onLogout={onLogout} />
+       <main className="main-content">{children}</main>
+       <Footer />
+     </div>
+   );
+ }
```

AJOUTÉ **src/components/templates/index.js** +3 −0

```
@@ -1,0 +1,3 @@
+ export { MainLayout } from "./MainLayout";
+ export { Header } from "./Header";
+ export { Footer } from "./Footer";
```

AJOUTÉ **src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.adapter.js** +5 −0

```
@@ -1,0 +1,5 @@
+ // Résultat simulé (succès ou échec) -> encadré à afficher
+ export const adaptDemoResult = (ok) =>
+   ok
+     ? { variant: "success", message: "Data loaded successfully!" }
+     : { variant: "error", message: "Failed to load data. Please try again." };
```

AJOUTÉ **src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.adapter.test.js** +12 −0

```
@@ -1,0 +1,12 @@
+ import { describe, it, expect } from "vitest";
+ import { adaptDemoResult } from "./ApiDemoScreen.adapter";
+
+ describe("adaptDemoResult", () => {
+   it("succès -> encadré vert", () => {
+     expect(adaptDemoResult(true)).toEqual({ variant: "success", message: "Data loaded succe
ssfully!" });
+   });
+
+   it("échec -> encadré rouge", () => {
+     expect(adaptDemoResult(false)).toEqual({ variant: "error", message: "Failed to load dat
a. Please try again." });
+   });
+ });
```

AJOUTÉ **src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.scene.jsx** +31 −0

```
@@ -1,0 +1,31 @@
+ import { Button } from "@/components/atoms";
+ import { StatusMessage } from "@/components/molecules";
+
+ export function ApiDemoScreenScene({ isLoading, result, onSimulate, onCrash }) {
+   return (
+     <div className="container">
+       <h1 className="page-title">API Demo</h1>
+
+       <div className="card api-demo-card">
+         <p className="subtitle">Test loading states and error messages</p>
+
+         <div className="button-group">
+           <Button variant="success" onClick={() => onSimulate(true)} disabled={isLoading}>
+             Simulate Success
+           </Button>
+           <Button variant="danger" onClick={() => onSimulate(false)} disabled={isLoading}>
+             Simulate Error
+           </Button>
+         </div>
```

```
20 +
21 +         <div className="button-group">
22 +           <Button onClick={onCrash}>Crash the page (Error Boundary)</Button>
23 +         </div>
24 +
25 +         {isLoading && <StatusMessage variant="loading">Loading...</StatusMessage>}
26 +
27 +         {result && <StatusMessage variant={result.variant}>{result.message}</StatusMessage
   >}
28 +       </div>
29 +     </div>
30 +   );
31 + }
```

AJOUTÉ **src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.view.jsx** +28 −0

```
   @@ -1,0 +1,28 @@
1  + import { useState } from "react";
2  + import { adaptDemoResult } from "./ApiDemoScreen.adapter";
3  + import { ApiDemoScreenScene } from "./ApiDemoScreen.scene";
4  +
5  + const SIMULATED_DELAY_MS = 2000;
6  +
7  + // Une page pour voir les 3 états d'un appel API : loading, succès, erreur
8  + export function ApiDemoScreenView() {
9  +   const [isLoading, setIsLoading] = useState(false);
10 +   const [result, setResult] = useState(null);
11 +   const [crash, setCrash] = useState(false);
12 +
13 +   // Pour tester l'Error Boundary : une erreur pendant le render
14 +   if (crash) {
15 +     throw new Error("Crash volontaire pour tester l'Error Boundary");
16 +   }
17 +
18 +   const simulate = (ok) => {
19 +     setResult(null);
20 +     setIsLoading(true);
21 +     setTimeout(() => {
22 +       setIsLoading(false);
23 +       setResult(adaptDemoResult(ok));
24 +     }, SIMULATED_DELAY_MS);
25 +   };
26 +
27 +   return <ApiDemoScreenScene isLoading={isLoading} result={result} onSimulate={simulate} on
   Crash={() => setCrash(true)} />;
28 + }
```

AJOUTÉ **src/features/api-demo/screens/ApiDemoScreen/index.js** +1 −0

```
   @@ -1,0 +1,1 @@
1  + export { ApiDemoScreenView as ApiDemoScreen } from "./ApiDemoScreen.view";
```

AJOUTÉ **src/features/auth/api/authApi.js** +5 −0

```
   @@ -1,0 +1,5 @@
1  + import axiosQuery from "@/lib/axiosQuery";
2  +
3  + // Login : l'API renvoie { session (le token), userDetails }
4  + export const login = (userName, password) =>
5  +   axiosQuery("post", "/api/login", { userName, password, verified: true, blocked: false, de
   lay: 0 });
```

AJOUTÉ **src/features/auth/hooks/index.js** +2 −0

```
   @@ -1,0 +1,2 @@
```

```
+ export { useCurrentUser } from "./useCurrentUser";
+ export { useLogin } from "./useLogin";
```

AJOUTÉ **src/features/auth/hooks/useCurrentUser.js** +29 −0

```
@@ -1,0 +1,29 @@
+ import { useEffect, useState } from "react";
+ import { useNavigate } from "react-router-dom";
+ import { storage, USER_UPDATED_EVENT } from "../utils/storage";
+
+ // L'utilisateur connecté (ou null) et la déconnexion
+ export const useCurrentUser = () => {
+   const navigate = useNavigate();
+   const [user, setUser] = useState(storage.getUser);
+
+   // On écoute les changements de l'utilisateur (login dans cet onglet ou dans un autre)
+   useEffect(() => {
+     const refreshUser = () => setUser(storage.getUser());
+     window.addEventListener("storage", refreshUser);
+     window.addEventListener(USER_UPDATED_EVENT, refreshUser);
+     return () => {
+       window.removeEventListener("storage", refreshUser);
+       window.removeEventListener(USER_UPDATED_EVENT, refreshUser);
+     };
+   }, []);
+
+   const logout = () => {
+     storage.removeToken();
+     storage.removeUser();
+     setUser(null);
+     navigate("/login");
+   };
+
+   return { user, logout };
+ };
```

AJOUTÉ **src/features/auth/hooks/useLogin.js** +22 −0

```
@@ -1,0 +1,22 @@
+ import { useMutation } from "@tanstack/react-query";
+ import { useNavigate } from "react-router-dom";
+ import { login } from "../api/authApi";
+ import { storage, USER_UPDATED_EVENT } from "../utils/storage";
+
+ // La mutation de login : enregistre la session puis retourne à l'accueil
+ export const useLogin = () => {
+   const navigate = useNavigate();
+
+   return useMutation({
+     mutationFn: ({ userName, password }) => login(userName, password),
+     onSuccess: (response) => {
+       if (response.session) storage.setToken(response.session);
+       if (response.userDetails) {
+         storage.setUser(response.userDetails);
+         // On prévient le Header que l'utilisateur a changé
+         window.dispatchEvent(new CustomEvent(USER_UPDATED_EVENT));
+       }
+       navigate("/");
+     },
+   });
+ };
```

AJOUTÉ **src/features/auth/screens/LoginScreen/LoginScreen.adapter.js** +7 −0

```
@@ -1,0 +1,7 @@
```

```
+ export const DEFAULT_LOGIN_ERROR = "Login failed. Please try again.";
+
+ // Erreur axios -> message lisible : message du serveur, sinon message technique, sinon message par défaut
+ export const getLoginErrorMessage = (error) => {
+   if (!error) return null;
+   return error.response?.data?.message || error.message || DEFAULT_LOGIN_ERROR;
+ };
```

AJOUTÉ **src/features/auth/screens/LoginScreen/LoginScreen.adapter.test.js** +22 −0

```
@@ -1,0 +1,22 @@
+ import { describe, it, expect } from "vitest";
+ import { getLoginErrorMessage, DEFAULT_LOGIN_ERROR } from "./LoginScreen.adapter";
+
+ describe("getLoginErrorMessage", () => {
+   it("renvoie null sans erreur", () => {
+     expect(getLoginErrorMessage(null)).toBeNull();
+     expect(getLoginErrorMessage(undefined)).toBeNull();
+   });
+
+   it("préfère le message renvoyé par le serveur", () => {
+     const error = { message: "Request failed with status code 401", response: { data: { message: "Wrong password" } } };
+     expect(getLoginErrorMessage(error)).toBe("Wrong password");
+   });
+
+   it("utilise le message de l'erreur sinon", () => {
+     expect(getLoginErrorMessage({ message: "Network Error" })).toBe("Network Error");
+   });
+
+   it("utilise le message par défaut en dernier recours", () => {
+     expect(getLoginErrorMessage({})).toBe(DEFAULT_LOGIN_ERROR);
+   });
+ });
```

AJOUTÉ **src/features/auth/screens/LoginScreen/LoginScreen.scene.jsx** +13 −0

```
@@ -1,0 +1,13 @@
+ import { LoginForm } from "@/components/organisms";
+
+ export function LoginScreenScene({ onSubmit, isPending, errorMessage }) {
+   return (
+     <div className="login-container">
+       <div className="card login-card">
+         <h1 className="login-title">Welcome Back</h1>
+         <p className="subtitle">Sign in to your account</p>
+         <LoginForm onSubmit={onSubmit} isPending={isPending} errorMessage={errorMessage} />
+       </div>
+     </div>
+   );
+ }
```

AJOUTÉ **src/features/auth/screens/LoginScreen/LoginScreen.view.jsx** +15 −0

```
@@ -1,0 +1,15 @@
+ import { useLogin } from "../../hooks";
+ import { getLoginErrorMessage } from "./LoginScreen.adapter";
+ import { LoginScreenScene } from "./LoginScreen.scene";
+
+ export function LoginScreenView() {
+   const loginMutation = useLogin();
+
+   return (
+     <LoginScreenScene
```

```
+       onSubmit={(credentials) => loginMutation.mutate(credentials)}
+       isPending={loginMutation.isPending}
+       errorMessage={getLoginErrorMessage(loginMutation.error)}
+     />
+   );
+ }
```

AJOUTÉ **src/features/auth/screens/LoginScreen/index.js** +1 −0

```
@@ -1,0 +1,1 @@
+ export { LoginScreenView as LoginScreen } from "./LoginScreen.view";
```

AJOUTÉ **src/features/auth/utils/authLoader.js** +19 −0

```
@@ -1,0 +1,19 @@
+ import { redirect } from "react-router-dom";
+ import { storage } from "./storage";
+
+ // Loader de /favorites : pas de token => redirection vers /login (avant même d'afficher la page)
+ export const requireAuth = () => {
+   const token = storage.getToken();
+   if (!token) {
+     throw redirect("/login");
+   }
+   return { token };
+ };
+
+ // Loader de /login : déjà connecté => retour à l'accueil
+ export const redirectIfAuthenticated = () => {
+   if (storage.getToken()) {
+     throw redirect("/");
+   }
+   return null;
+ };
```

AJOUTÉ **src/features/auth/utils/authLoader.test.js** +51 −0

```
@@ -1,0 +1,51 @@
+ import { describe, it, expect, vi, beforeEach } from "vitest";
+ import { requireAuth, redirectIfAuthenticated } from "./authLoader";
+ import { storage } from "./storage";
+
+ // On remplace le stockage : pas besoin de localStorage (environnement node)
+ vi.mock("./storage", () => ({
+   storage: { getToken: vi.fn() },
+ }));
+
+ // Un loader qui redirige "throw" une Response : on la récupère pour l'inspecter
+ const catchThrown = (fn) => {
+   try {
+     fn();
+   } catch (thrown) {
+     return thrown;
+   }
+   return undefined;
+ };
+
+ describe("requireAuth", () => {
+   beforeEach(() => vi.mocked(storage.getToken).mockReset());
+
+   it("redirige vers /login s'il n'y a pas de token", () => {
+     vi.mocked(storage.getToken).mockReturnValue(null);
+     const response = catchThrown(requireAuth);
+     expect(response).toBeInstanceOf(Response);
```

```
+     expect(response.status).toBe(302);
+     expect(response.headers.get("Location")).toBe("/login");
+   });
+
+   it("renvoie le token s'il existe", () => {
+     vi.mocked(storage.getToken).mockReturnValue("abc");
+     expect(requireAuth()).toEqual({ token: "abc" });
+   });
+ });
+
+ describe("redirectIfAuthenticated", () => {
+   beforeEach(() => vi.mocked(storage.getToken).mockReset());
+
+   it("redirige vers / si l'utilisateur est déjà connecté", () => {
+     vi.mocked(storage.getToken).mockReturnValue("abc");
+     const response = catchThrown(redirectIfAuthenticated);
+     expect(response).toBeInstanceOf(Response);
+     expect(response.headers.get("Location")).toBe("/");
+   });
+
+   it("renvoie null si personne n'est connecté", () => {
+     vi.mocked(storage.getToken).mockReturnValue(null);
+     expect(redirectIfAuthenticated()).toBeNull();
+   });
+ });
```

AJOUTÉ **src/features/auth/utils/storage.js** +19 −0

```
@@ -1,0 +1,19 @@
+ // Le token et l'utilisateur sont gardés dans le localStorage (la session survit au recharg
ement)
+ const TOKEN_KEY = "auth_token";
+ const USER_KEY = "auth_user";
+
+ // Événement envoyé après un login pour prévenir le Header (dans le même onglet)
+ export const USER_UPDATED_EVENT = "userUpdated";
+
+ export const storage = {
+   setToken: (token) => localStorage.setItem(TOKEN_KEY, token),
+   getToken: () => localStorage.getItem(TOKEN_KEY),
+   removeToken: () => localStorage.removeItem(TOKEN_KEY),
+
+   setUser: (user) => localStorage.setItem(USER_KEY, JSON.stringify(user)),
+   getUser: () => {
+     const user = localStorage.getItem(USER_KEY);
+     return user ? JSON.parse(user) : null;
+   },
+   removeUser: () => localStorage.removeItem(USER_KEY),
+ };
```

AJOUTÉ **src/features/cars/adapters/carAdapter.js** +30 −0

```
@@ -1,0 +1,30 @@
+ import { getColorHex } from "../utils/colorUtils";
+
+ // Texte affiché quand une valeur manque dans la réponse de l'API
+ export const MISSING_VALUE = "—";
+
+ const isMissing = (value) => value === undefined || value === null || value === "";
+ export const orMissing = (value) => (isMissing(value) ? MISSING_VALUE : value);
+
+ // Champs communs à la carte et à la page détails : DTO de l'API -> modèle pour l'UI
+ export const adaptCarBase = (dto = {}) => {
+   const isAvailable = Boolean(dto.availability);
+   return {
```

```
+     id: dto.id,
+     title: [dto.car, dto.car_model].filter((part) => !isMissing(part)).join(" ") || "Unknow
n car",
+     colorHex: getColorHex(dto.car_color),
+     colorName: orMissing(dto.car_color),
+     year: orMissing(dto.car_model_year),
+     vin: orMissing(dto.car_vin),
+     priceLabel: orMissing(dto.price),
+     isAvailable,
+     statusLabel: isAvailable ? "Available" : "Unavailable",
+   };
+ };
+
+ // Modèle d'une carte voiture (liste Home et Favoris)
+ export const adaptCarCard = (dto, isFavorite = false) => ({
+   ...adaptCarBase(dto),
+   detailsUrl: `/car/${dto?.id}`,
+   isFavorite,
+ });
```

AJOUTÉ **src/features/cars/adapters/carAdapter.test.js** +69 −0

```
@@ -1,0 +1,69 @@
+ import { describe, it, expect } from "vitest";
+ import { adaptCarCard, MISSING_VALUE } from "./carAdapter";
+ import { DEFAULT_COLOR } from "../utils/colorUtils";
+
+ const dto = {
+   id: 12,
+   car: "Audi",
+   car_model: "Q7",
+   car_color: "Red",
+   car_model_year: 2012,
+   car_vin: "WAUAF78E96A123456",
+   price: "$23,456.78",
+   availability: true,
+ };
+
+ describe("adaptCarCard", () => {
+   it("transforme le DTO de l'API en modèle pour la carte", () => {
+     expect(adaptCarCard(dto)).toEqual({
+       id: 12,
+       title: "Audi Q7",
+       colorHex: "#e74c3c",
+       colorName: "Red",
+       year: 2012,
+       vin: "WAUAF78E96A123456",
+       priceLabel: "$23,456.78",
+       isAvailable: true,
+       statusLabel: "Available",
+       detailsUrl: "/car/12",
+       isFavorite: false,
+     });
+   });
+
+   it("indique si la voiture est en favori", () => {
+     expect(adaptCarCard(dto, true).isFavorite).toBe(true);
+   });
+
+   it("marque une voiture indisponible", () => {
+     const car = adaptCarCard({ ...dto, availability: false });
+     expect(car.isAvailable).toBe(false);
+     expect(car.statusLabel).toBe("Unavailable");
+   });
+
```

```
+   it("résiste aux valeurs manquantes", () => {
+     const car = adaptCarCard({ id: 3 });
+     expect(car.title).toBe("Unknown car");
+     expect(car.colorHex).toBe(DEFAULT_COLOR);
+     expect(car.colorName).toBe(MISSING_VALUE);
+     expect(car.priceLabel).toBe(MISSING_VALUE);
+     expect(car.year).toBe(MISSING_VALUE);
+     expect(car.vin).toBe(MISSING_VALUE);
+     expect(car.isAvailable).toBe(false);
+     expect(car.statusLabel).toBe("Unavailable");
+     expect(car.detailsUrl).toBe("/car/3");
+   });
+
+   it("garde le titre lisible si seul le modèle manque", () => {
+     expect(adaptCarCard({ ...dto, car_model: null }).title).toBe("Audi");
+   });
+
+   it("couleur inconnue -> gris, mais garde son nom", () => {
+     const car = adaptCarCard({ ...dto, car_color: "Teal" });
+     expect(car.colorHex).toBe(DEFAULT_COLOR);
+     expect(car.colorName).toBe("Teal");
+   });
+
+   it("prix vide -> tiret", () => {
+     expect(adaptCarCard({ ...dto, price: "" }).priceLabel).toBe(MISSING_VALUE);
+   });
+ });
```

AJOUTÉ **src/features/cars/api/carsApi.js** +7 −0

```
@@ -1,0 +1,7 @@
+ import axiosQuery from "@/lib/axiosQuery";
+
+ // Liste des voitures d'une année : l'API renvoie { Cars: [...] }
+ export const fetchCarsByYear = (year) => axiosQuery("get", `/api/cars/year/${year}`);
+
+ // Détails d'une voiture : l'API renvoie { Car: {...} }
+ export const fetchCarById = (id) => axiosQuery("get", `/api/cars/${id}`);
```

AJOUTÉ **src/features/cars/hooks/index.js** +2 −0

```
@@ -1,0 +1,2 @@
+ export { useCars } from "./useCars";
+ export { useCarDetails } from "./useCarDetails";
```

AJOUTÉ **src/features/cars/hooks/queryKeys.js** +5 −0

```
@@ -1,0 +1,5 @@
+ // Les clés React Query de la feature, centralisées pour éviter les fautes de frappe
+ export const QUERY_KEYS = {
+   CARS_BY_YEAR: "cars",
+   CAR_DETAILS: "car",
+ };
```

AJOUTÉ **src/features/cars/hooks/useCarDetails.js** +11 −0

```
@@ -1,0 +1,11 @@
+ import { useQuery } from "@tanstack/react-query";
+ import { fetchCarById } from "../api/carsApi";
+ import { QUERY_KEYS } from "./queryKeys";
+
+ // L'id fait partie de la clé : chaque voiture a son propre cache
+ export const useCarDetails = (id) =>
+   useQuery({
+     queryKey: [QUERY_KEYS.CAR_DETAILS, id],
```

```
+     queryFn: () => fetchCarById(id),
+     enabled: Boolean(id),
+   });
```

AJOUTÉ **src/features/cars/hooks/useCars.js** +10 −0

```
@@ -1,0 +1,10 @@
+ import { useQuery } from "@tanstack/react-query";
+ import { fetchCarsByYear } from "../api/carsApi";
+ import { QUERY_KEYS } from "./queryKeys";
+
+ // L'année fait partie de la clé : chaque année a son propre cache
+ export const useCars = (year) =>
+   useQuery({
+     queryKey: [QUERY_KEYS.CARS_BY_YEAR, year],
+     queryFn: () => fetchCarsByYear(year),
+   });
```

AJOUTÉ **src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.adapter.js** +14 −0

```
@@ -1,0 +1,14 @@
+ import { adaptCarBase, orMissing } from "../../adapters/carAdapter";
+
+ // Réponse de l'API { Car: {...} } -> modèle de la page détails (null si la voiture est absente)
+ export const adaptCarDetails = (response, favoriteIds = []) => {
+   const dto = response?.Car;
+   if (!dto) return null;
+
+   return {
+     ...adaptCarBase(dto),
+     brand: orMissing(dto.car),
+     model: orMissing(dto.car_model),
+     isFavorite: favoriteIds.includes(dto.id),
+   };
+ };
```

AJOUTÉ **src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.adapter.test.js** +53 −0

```
@@ -1,0 +1,53 @@
+ import { describe, it, expect } from "vitest";
+ import { adaptCarDetails } from "./CarDetailsScreen.adapter";
+ import { MISSING_VALUE } from "../../adapters/carAdapter";
+ import { DEFAULT_COLOR } from "../../utils/colorUtils";
+
+ const dto = {
+   id: 7,
+   car: "Ford",
+   car_model: "Focus",
+   car_color: "Green",
+   car_model_year: 2012,
+   car_vin: "1FAHP3F2XCL123456",
+   price: "$12,345.00",
+   availability: false,
+ };
+
+ describe("adaptCarDetails", () => {
+   it("transforme la réponse de l'API en modèle de la page détails", () => {
+     expect(adaptCarDetails({ Car: dto })).toEqual({
+       id: 7,
+       title: "Ford Focus",
+       brand: "Ford",
+       model: "Focus",
+       colorHex: "#2ecc71",
+       colorName: "Green",
```

```
+       year: 2012,
+       vin: "1FAHP3F2XCL123456",
+       priceLabel: "$12,345.00",
+       isAvailable: false,
+       statusLabel: "Unavailable",
+       isFavorite: false,
+     });
+   });
+
+   it("indique si la voiture est en favori", () => {
+     expect(adaptCarDetails({ Car: dto }, [7]).isFavorite).toBe(true);
+   });
+
+   it("renvoie null si la voiture est absente", () => {
+     expect(adaptCarDetails(undefined)).toBeNull();
+     expect(adaptCarDetails({})).toBeNull();
+     expect(adaptCarDetails({ Car: null })).toBeNull();
+   });
+
+   it("résiste aux champs manquants", () => {
+     const car = adaptCarDetails({ Car: { id: 8 } });
+     expect(car.brand).toBe(MISSING_VALUE);
+     expect(car.model).toBe(MISSING_VALUE);
+     expect(car.vin).toBe(MISSING_VALUE);
+     expect(car.priceLabel).toBe(MISSING_VALUE);
+     expect(car.colorHex).toBe(DEFAULT_COLOR);
+   });
+ });
```

AJOUTÉ **src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.scene.jsx** +49 −0

```
@@ -1,0 +1,49 @@
+ import { Link } from "react-router-dom";
+ import { BackIcon, ColorDot, Price, StatusBadge } from "@/components/atoms";
+ import { DetailItem, FavoriteButton } from "@/components/molecules";
+
+ // Présentation pure de la fiche voiture
+ export function CarDetailsScreenScene({ car, onToggleFavorite }) {
+   return (
+     <div className="container">
+       <Link to="/" className="back-link">
+         <BackIcon />
+         Back to Catalog
+       </Link>
+
+       <div className="card detail-card">
+         <div className="detail-header">
+           <h1 className="detail-title">{car.title}</h1>
+           <FavoriteButton isActive={car.isFavorite} onToggle={onToggleFavorite} />
+         </div>
+
+         <div className="detail-grid">
+           <DetailItem label="Brand">
+             <span className="detail-value">{car.brand}</span>
+           </DetailItem>
+           <DetailItem label="Model">
+             <span className="detail-value">{car.model}</span>
+           </DetailItem>
+           <DetailItem label="Color">
+             <div className="color-detail">
+               <ColorDot color={car.colorHex} />
+               <span className="detail-value">{car.colorName}</span>
+             </div>
+           </DetailItem>
+           <DetailItem label="Year">
```

```
34 +              <span className="detail-value">{car.year}</span>
35 +            </DetailItem>
36 +            <DetailItem label="VIN">
37 +              <span className="detail-value">{car.vin}</span>
38 +            </DetailItem>
39 +            <DetailItem label="Price">
40 +              <Price value={car.priceLabel} className="detail-value" />
41 +            </DetailItem>
42 +            <DetailItem label="Availability">
43 +              <StatusBadge isAvailable={car.isAvailable} label={car.statusLabel} className="d
   etail-value" />
44 +            </DetailItem>
45 +          </div>
46 +        </div>
47 +      </div>
48 +    );
49 + }
```

AJOUTÉ `src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.view.jsx` +28 −0

```
@@ -1,0 +1,28 @@
1  + import { useParams } from "react-router-dom";
2  + import { Loading } from "@/components/molecules";
3  + import { useFavorites } from "@/features/favorites/hooks";
4  + import { useCarDetails } from "../../hooks";
5  + import { adaptCarDetails } from "./CarDetailsScreen.adapter";
6  + import { CarDetailsScreenScene } from "./CarDetailsScreen.scene";
7  +
8  + export function CarDetailsScreenView() {
9  +   const { id } = useParams();
10 +   const { data, isLoading, error } = useCarDetails(id);
11 +   const { favoriteIds, toggleFavorite } = useFavorites();
12 +
13 +   if (isLoading) return <Loading text="Loading car details..." />;
14 +
15 +   const car = adaptCarDetails(data, favoriteIds);
16 +
17 +   // Erreur réseau ou voiture absente : même message
18 +   if (error || !car) {
19 +     return (
20 +       <div className="container">
21 +         <p>Car not found</p>
22 +       </div>
23 +     );
24 +   }
25 +
26 +   // Dans Redux on stocke l'objet API brut, pas le modèle adapté
27 +   return <CarDetailsScreenScene car={car} onToggleFavorite={() => toggleFavorite(data.Car)}
   />;
28 + }
```

AJOUTÉ `src/features/cars/screens/CarDetailsScreen/index.js` +1 −0

```
@@ -1,0 +1,1 @@
1  + export { CarDetailsScreenView as CarDetailsScreen } from "./CarDetailsScreen.view";
```

AJOUTÉ `src/features/cars/screens/HomeScreen/HomeScreen.adapter.js` +15 −0

```
@@ -1,0 +1,15 @@
1  + import { adaptCarCard } from "../../adapters/carAdapter";
2  +
3  + // La page d'accueil n'affiche que les 6 premières voitures
4  + export const HOME_CARS_LIMIT = 6;
5  +
6  + const getCarsList = (response) => (Array.isArray(response?.Cars) ? response.Cars : []);
```

```
+
+ // Réponse de l'API { Cars: [...] } -> liste de cartes prêtes à afficher
+ export const adaptHomeCars = (response, favoriteIds = []) =>
+   getCarsList(response)
+     .slice(0, HOME_CARS_LIMIT)
+     .map((dto) => adaptCarCard(dto, favoriteIds.includes(dto.id)));
+
+ // Retrouve l'objet API brut d'une voiture (c'est lui qu'on stocke dans les favoris)
+ export const findCarById = (response, id) => getCarsList(response).find((car) => car.id ===
id);
```

AJOUTÉ **src/features/cars/screens/HomeScreen/HomeScreen.adapter.test.js** +55 −0

```
@@ -1,0 +1,55 @@
+ import { describe, it, expect } from "vitest";
+ import { adaptHomeCars, findCarById, HOME_CARS_LIMIT } from "./HomeScreen.adapter";
+
+ const makeCar = (id) => ({
+   id,
+   car: "Audi",
+   car_model: `Model ${id}`,
+   car_color: "Blue",
+   car_model_year: 2012,
+   car_vin: `VIN${id}`,
+   price: "$10,000.00",
+   availability: id % 2 === 0,
+ });
+
+ const response = { Cars: Array.from({ length: 10 }, (_, i) => makeCar(i + 1)) };
+
+ describe("adaptHomeCars", () => {
+   it("limite la liste à 6 voitures", () => {
+     const cars = adaptHomeCars(response);
+     expect(HOME_CARS_LIMIT).toBe(6);
+     expect(cars).toHaveLength(6);
+     expect(cars.map((car) => car.id)).toEqual([1, 2, 3, 4, 5, 6]);
+   });
+
+   it("garde toutes les voitures s'il y en a moins de 6", () => {
+     expect(adaptHomeCars({ Cars: [makeCar(1), makeCar(2)] })).toHaveLength(2);
+   });
+
+   it("transforme chaque voiture en modèle de carte", () => {
+     const [first] = adaptHomeCars(response);
+     expect(first).toMatchObject({ id: 1, title: "Audi Model 1", colorHex: "#3498db", status
Label: "Unavailable", detailsUrl: "/car/1" });
+   });
+
+   it("marque les voitures en favori", () => {
+     const cars = adaptHomeCars(response, [2, 5]);
+     expect(cars.filter((car) => car.isFavorite).map((car) => car.id)).toEqual([2, 5]);
+   });
+
+   it("renvoie une liste vide si la réponse est vide ou mal formée", () => {
+     expect(adaptHomeCars(undefined)).toEqual([]);
+     expect(adaptHomeCars({})).toEqual([]);
+     expect(adaptHomeCars({ Cars: null })).toEqual([]);
+   });
+ });
+
+ describe("findCarById", () => {
+   it("retrouve l'objet API brut", () => {
+     expect(findCarById(response, 3)).toEqual(makeCar(3));
+   });
+
+
```

```
+   it("renvoie undefined si la voiture n'existe pas", () => {
+     expect(findCarById(response, 99)).toBeUndefined();
+     expect(findCarById(undefined, 1)).toBeUndefined();
+   });
+ });
```

AJOUTÉ **src/features/cars/screens/HomeScreen/HomeScreen.scene.jsx** +11 −0

```
@@ -1,0 +1,11 @@
+ import { CarGrid } from "@/components/organisms";
+
+ // Présentation pure : reçoit les cartes déjà adaptées
+ export function HomeScreenScene({ cars, onToggleFavorite }) {
+   return (
+     <div className="container">
+       <h1 className="page-title">Car Catalog</h1>
+       <CarGrid cars={cars} onToggleFavorite={onToggleFavorite} />
+     </div>
+   );
+ }
```

AJOUTÉ **src/features/cars/screens/HomeScreen/HomeScreen.view.jsx** +23 −0

```
@@ -1,0 +1,23 @@
+ import { Loading, ErrorMessage } from "@/components/molecules";
+ import { useFavorites } from "@/features/favorites/hooks";
+ import { useCars } from "../../hooks";
+ import { adaptHomeCars, findCarById } from "./HomeScreen.adapter";
+ import { HomeScreenScene } from "./HomeScreen.scene";
+
+ const CATALOG_YEAR = 2012;
+
+ // Conteneur : récupère les données, gère chargement / erreur, adapte, puis passe la main à la Scene
+ export function HomeScreenView() {
+   const { data, isLoading, error } = useCars(CATALOG_YEAR);
+   const { favoriteIds, toggleFavorite } = useFavorites();
+
+   if (isLoading) return <Loading text="Loading cars..." />;
+   if (error) return <ErrorMessage message={`Error loading cars: ${error.message}`} />;
+
+   const handleToggleFavorite = (id) => {
+     const car = findCarById(data, id);
+     if (car) toggleFavorite(car);
+   };
+
+   return <HomeScreenScene cars={adaptHomeCars(data, favoriteIds)} onToggleFavorite={handleToggleFavorite} />;
+ }
```

AJOUTÉ **src/features/cars/screens/HomeScreen/index.js** +1 −0

```
@@ -1,0 +1,1 @@
+ export { HomeScreenView as HomeScreen } from "./HomeScreen.view";
```

AJOUTÉ **src/features/cars/utils/colorUtils.js** +26 −0

```
@@ -1,0 +1,26 @@
+ // L'API donne un nom de couleur : on le transforme en code couleur pour la pastille.
+ const colorMap = {
+   Red: "#e74c3c",
+   Blue: "#3498db",
+   Green: "#2ecc71",
+   Yellow: "#f1c40f",
+   Orange: "#f39c12",
```

```
+   Purple: "#9b59b6",
+   Pink: "#e91e63",
+   Violet: "#9b59b6",
+   Turquoise: "#1abc9c",
+   Crimson: "#dc143c",
+   Indigo: "#4b0082",
+   Mauv: "#e0b0ff",
+   Fuscia: "#ff00ff",
+   Khaki: "#f0e68c",
+   Maroon: "#800000",
+   Aquamarine: "#7fffd4",
+   Puce: "#cc8899",
+   Goldenrod: "#daa520",
+ };
+
+ // Couleur inconnue ou absente : gris
+ export const DEFAULT_COLOR = "#95a5a6";
+
+ export const getColorHex = (colorName) => colorMap[colorName] || DEFAULT_COLOR;
```

AJOUTÉ **src/features/cars/utils/colorUtils.test.js** **+23 −0**

```
@@ -1,0 +1,23 @@
+ import { describe, it, expect } from "vitest";
+ import { getColorHex, DEFAULT_COLOR } from "./colorUtils";
+
+ describe("getColorHex", () => {
+   it("renvoie le code couleur d'une couleur connue", () => {
+     expect(getColorHex("Red")).toBe("#e74c3c");
+     expect(getColorHex("Goldenrod")).toBe("#daa520");
+   });
+
+   it("renvoie le gris par défaut pour une couleur inconnue", () => {
+     expect(getColorHex("Teal")).toBe(DEFAULT_COLOR);
+   });
+
+   it("renvoie le gris par défaut si la couleur est absente", () => {
+     expect(getColorHex(undefined)).toBe(DEFAULT_COLOR);
+     expect(getColorHex(null)).toBe(DEFAULT_COLOR);
+     expect(getColorHex("")).toBe(DEFAULT_COLOR);
+   });
+
+   it("respecte la casse de l'API", () => {
+     expect(getColorHex("red")).toBe(DEFAULT_COLOR);
+   });
+ });
```

AJOUTÉ **src/features/errors/screens/ErrorScreen/ErrorScreen.adapter.js** **+4 −0**

```
@@ -1,0 +1,4 @@
+ export const DEFAULT_ERROR_MESSAGE = "An unexpected error occurred";
+
+ // L'erreur levée par une route (Error, Response, valeur quelconque) -> message à afficher
+ export const getErrorMessage = (error) => error?.message || DEFAULT_ERROR_MESSAGE;
```

AJOUTÉ **src/features/errors/screens/ErrorScreen/ErrorScreen.adapter.test.js** **+13 −0**

```
@@ -1,0 +1,13 @@
+ import { describe, it, expect } from "vitest";
+ import { getErrorMessage, DEFAULT_ERROR_MESSAGE } from "./ErrorScreen.adapter";
+
+ describe("getErrorMessage", () => {
+   it("renvoie le message de l'erreur", () => {
+     expect(getErrorMessage(new Error("Boom"))).toBe("Boom");
+   });
```

```
+
+   it("renvoie le message par défaut sinon", () => {
+     expect(getErrorMessage(undefined)).toBe(DEFAULT_ERROR_MESSAGE);
+     expect(getErrorMessage({ status: 500 })).toBe(DEFAULT_ERROR_MESSAGE);
+   });
+ });
```

AJOUTÉ **src/features/errors/screens/ErrorScreen/ErrorScreen.scene.jsx** +15 −0

```
@@ -1,0 +1,15 @@
+ import { AlertIcon, Button, ButtonLink } from "@/components/atoms";
+
+ export function ErrorScreenScene({ message, onRetry }) {
+   return (
+     <div className="center-page error-content">
+       <AlertIcon size={64} />
+       <h1 className="error-title">Something went wrong!</h1>
+       <p className="muted">{message}</p>
+       <div className="button-group">
+         <Button onClick={onRetry}>Try again</Button>
+         <ButtonLink to="/">Go Home</ButtonLink>
+       </div>
+     </div>
+   );
+ }
```

AJOUTÉ **src/features/errors/screens/ErrorScreen/ErrorScreen.view.jsx** +14 −0

```
@@ -1,0 +1,14 @@
+ import { useNavigate, useRouteError } from "react-router-dom";
+ import { getErrorMessage } from "./ErrorScreen.adapter";
+ import { ErrorScreenScene } from "./ErrorScreen.scene";
+
+ // errorElement de React Router : une erreur dans une page affiche cet écran au lieu de cas
ser l'application
+ export function ErrorScreenView() {
+   const error = useRouteError();
+   const navigate = useNavigate();
+
+   console.error("ErrorBoundary:", error);
+
+   // navigate(0) recharge la page courante
+   return <ErrorScreenScene message={getErrorMessage(error)} onRetry={() => navigate(0)} />;
+ }
```

AJOUTÉ **src/features/errors/screens/ErrorScreen/index.js** +1 −0

```
@@ -1,0 +1,1 @@
+ export { ErrorScreenView as ErrorScreen } from "./ErrorScreen.view";
```

AJOUTÉ **src/features/errors/screens/NotFoundScreen/NotFoundScreen.scene.jsx** +12 −0

```
@@ -1,0 +1,12 @@
+ import { ButtonLink } from "@/components/atoms";
+
+ // Page 404 : aucune donnée, donc pas de View ni d'adapter, la Scene suffit
+ export function NotFoundScreenScene() {
+   return (
+     <div className="center-page">
+       <h1 className="not-found-title">404</h1>
+       <p className="not-found-text">Page not found</p>
+       <ButtonLink to="/">Go Home</ButtonLink>
+     </div>
+   );
+ }
```

AJOUTÉ **src/features/errors/screens/NotFoundScreen/index.js** +1 −0

```
@@ -1,0 +1,1 @@
+ export { NotFoundScreenScene as NotFoundScreen } from "./NotFoundScreen.scene";
```

AJOUTÉ **src/features/favorites/favoritesSlice.js** +28 −0

```
@@ -1,0 +1,28 @@
+ import { createSlice } from "@reduxjs/toolkit";
+
+ const initialState = {
+   favorites: [],
+ };
+
+ const favoritesSlice = createSlice({
+   name: "favorites",
+   initialState,
+   reducers: {
+     // Si la voiture est déjà en favori on la retire, sinon on l'ajoute
+     toggleFavorite: (state, action) => {
+       const carExists = state.favorites.some((car) => car.id === action.payload.id);
+       if (carExists) {
+         state.favorites = state.favorites.filter((car) => car.id !== action.payload.id);
+       } else {
+         state.favorites.push(action.payload);
+       }
+     },
+   },
+ });
+
+ export const { toggleFavorite } = favoritesSlice.actions;
+
+ // Selectors
+ export const selectFavorites = (state) => state.favorites.favorites;
+
+ export default favoritesSlice.reducer;
```

AJOUTÉ **src/features/favorites/favoritesSlice.test.js** +62 −0

```
@@ -1,0 +1,62 @@
+ import { describe, it, expect } from 'vitest';
+ import favoritesReducer, { toggleFavorite } from './favoritesSlice';
+
+ describe('favoritesSlice', () => {
+   const mockCar = {
+     id: 1,
+     car: 'Tesla',
+     car_model: 'Model S',
+     car_color: 'Red',
+     car_model_year: 2024,
+     car_vin: 'TEST123',
+     price: '$79,990',
+     availability: true,
+   };
+
+   const mockCar2 = {
+     id: 2,
+     car: 'BMW',
+     car_model: 'i8',
+     car_color: 'Blue',
+     car_model_year: 2023,
+     car_vin: 'TEST456',
+     price: '$147,500',
+     availability: false,
+   };
```

```
+
+   it('should return initial state', () => {
+     expect(favoritesReducer(undefined, { type: 'unknown' })).toEqual({
+       favorites: [],
+     });
+   });
+
+   it('should add a car to favorites', () => {
+     const state = favoritesReducer(undefined, toggleFavorite(mockCar));
+     expect(state.favorites).toHaveLength(1);
+     expect(state.favorites[0]).toEqual(mockCar);
+   });
+
+   it('should remove a car from favorites when toggled again', () => {
+     const initialState = { favorites: [mockCar] };
+     const state = favoritesReducer(initialState, toggleFavorite(mockCar));
+     expect(state.favorites).toHaveLength(0);
+   });
+
+   it('should add multiple cars to favorites', () => {
+     let state = favoritesReducer(undefined, toggleFavorite(mockCar));
+     state = favoritesReducer(state, toggleFavorite(mockCar2));
+
+     expect(state.favorites).toHaveLength(2);
+     expect(state.favorites).toContainEqual(mockCar);
+     expect(state.favorites).toContainEqual(mockCar2);
+   });
+
+   it('should remove specific car without affecting others', () => {
+     const initialState = { favorites: [mockCar, mockCar2] };
+     const state = favoritesReducer(initialState, toggleFavorite(mockCar));
+
+     expect(state.favorites).toHaveLength(1);
+     expect(state.favorites[0]).toEqual(mockCar2);
+   });
+ });
+
```

AJOUTÉ **src/features/favorites/hooks/index.js** **+1 −0**

```
@@ -1,0 +1,1 @@
+ export { useFavorites } from "./useFavorites";
```

AJOUTÉ **src/features/favorites/hooks/useFavorites.js** **+14 −0**

```
@@ -1,0 +1,14 @@
+ import { useMemo } from "react";
+ import { useDispatch, useSelector } from "react-redux";
+ import { selectFavorites, toggleFavorite as toggleFavoriteAction } from "../favoritesSlic
e";
+
+ // Les écrans n'utilisent jamais useSelector / useDispatch directement : tout passe par ce
hook
+ export const useFavorites = () => {
+   const dispatch = useDispatch();
+   const favorites = useSelector(selectFavorites);
+   const favoriteIds = useMemo(() => favorites.map((car) => car.id), [favorites]);
+
+   const toggleFavorite = (car) => dispatch(toggleFavoriteAction(car));
+
+   return { favorites, favoriteIds, toggleFavorite };
+ };
```

AJOUTÉ **src/features/favorites/screens/FavoritesScreen/FavoritesScreen.adapter.js** **+4 −0**

```
@@ -1,0 +1,4 @@
+ import { adaptCarCard } from "@/features/cars/adapters/carAdapter";
+
+ // Les favoris (objets API bruts stockés dans Redux) -> cartes prêtes à afficher, toutes en favori
+ export const adaptFavoriteCars = (favorites = []) => favorites.map((dto) => adaptCarCard(dto, true));
```

AJOUTÉ **src/features/favorites/screens/FavoritesScreen/FavoritesScreen.adapter.test.js** +20 −0

```
@@ -1,0 +1,20 @@
+ import { describe, it, expect } from "vitest";
+ import { adaptFavoriteCars } from "./FavoritesScreen.adapter";
+
+ describe("adaptFavoriteCars", () => {
+   it("transforme chaque favori en carte marquée comme favorite", () => {
+     const cars = adaptFavoriteCars([
+       { id: 1, car: "BMW", car_model: "X5", price: "$50,000.00", availability: true },
+       { id: 2, car: "Kia", car_model: "Rio" },
+     ]);
+     expect(cars).toHaveLength(2);
+     expect(cars.every((car) => car.isFavorite)).toBe(true);
+     expect(cars[0]).toMatchObject({ title: "BMW X5", priceLabel: "$50,000.00", statusLabel: "Available" });
+     expect(cars[1]).toMatchObject({ title: "Kia Rio", priceLabel: "—", statusLabel: "Unavailable" });
+   });
+
+   it("renvoie une liste vide sans favoris", () => {
+     expect(adaptFavoriteCars([])).toEqual([]);
+     expect(adaptFavoriteCars()).toEqual([]);
+   });
+ });
```

AJOUTÉ **src/features/favorites/screens/FavoritesScreen/FavoritesScreen.scene.jsx** +14 −0

```
@@ -1,0 +1,14 @@
+ import { CarGrid } from "@/components/organisms";
+
+ export function FavoritesScreenScene({ cars, onToggleFavorite }) {
+   return (
+     <div className="container">
+       <h1 className="page-title">My Favorites</h1>
+       {cars.length === 0 ? (
+         <p className="subtitle">No favorites yet. Add some cars from the home page!</p>
+       ) : (
+         <CarGrid cars={cars} onToggleFavorite={onToggleFavorite} />
+       )}
+     </div>
+   );
+ }
```

AJOUTÉ **src/features/favorites/screens/FavoritesScreen/FavoritesScreen.view.jsx** +15 −0

```
@@ -1,0 +1,15 @@
+ import { useFavorites } from "../../hooks";
+ import { adaptFavoriteCars } from "./FavoritesScreen.adapter";
+ import { FavoritesScreenScene } from "./FavoritesScreen.scene";
+
+ // Pas d'appel API ici : les données viennent du store Redux
+ export function FavoritesScreenView() {
+   const { favorites, toggleFavorite } = useFavorites();
+
+   const handleToggleFavorite = (id) => {
```

```
+     const car = favorites.find((favorite) => favorite.id === id);
+     if (car) toggleFavorite(car);
+   };
+
+   return <FavoritesScreenScene cars={adaptFavoriteCars(favorites)} onToggleFavorite={handle
ToggleFavorite} />;
+ }
```

AJOUTÉ `src/features/favorites/screens/FavoritesScreen/index.js` +1 −0

```
@@ -1,0 +1,1 @@
+ export { FavoritesScreenView as FavoritesScreen } from "./FavoritesScreen.view";
```

SUPPRIMÉ `src/layouts/MainLayout.jsx` +0 −15

```
@@ -1,15 +1,0 @@
- import { Outlet } from "react-router-dom";
- import Header from "../components/Header";
- import Footer from "../components/Footer";
-
- export default function MainLayout() {
-   return (
-     <div className="app-layout">
-       <Header />
-       <main className="main-content">
-         <Outlet />
-       </main>
-       <Footer />
-     </div>
-   );
- }
```

MODIFIÉ `src/lib/axiosQuery.js` +2 −3

```
@@ -1,6 +1,6 @@
  import axios from "axios";
- import { BASE_URL } from "../api/config";
- import { storage } from "../utils/storage";
+ import { BASE_URL } from "./config";
+ import { storage } from "@/features/auth/utils/storage";

  const axiosInstance = axios.create({
    baseURL: BASE_URL,
@@ -26,5 +26,4 @@
    return response.data;
  };

- export { axiosInstance };
  export default axiosQuery;
```

AJOUTÉ `src/lib/config.js` +2 −0

```
@@ -1,0 +1,2 @@
+ // Les requêtes passent par le proxy Vite (/api) : pas besoin d'URL complète en dev
+ export const BASE_URL = "";
```

MODIFIÉ `src/main.jsx` +5 −12

```
@@ -1,20 +1,13 @@
  import { StrictMode } from "react";
  import { createRoot } from "react-dom/client";
- import { Provider } from "react-redux";
- import { QueryClientProvider } from "@tanstack/react-query";
- import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
```

```
6       - import { queryClient } from "./lib/queryClient";
7       - import { store } from "./store/store";
8       - import App from "./App";
        3 + import AppProviders from "./app/providers";
        4 + import App from "./app/App";
9       5   import "./styles.css";
10      6
11      7   createRoot(document.getElementById("root")).render(
12      8     <StrictMode>
13      -     <Provider store={store}>
14      -       <QueryClientProvider client={queryClient}>
15      -         <App />
16      -         <ReactQueryDevtools initialIsOpen={false} />
17      -       </QueryClientProvider>
18      -     </Provider>
        9 +     <AppProviders>
        10 +       <App />
        11 +     </AppProviders>
19      12    </StrictMode>
20      13  );
```

SUPPRIMÉ **src/pages/ApiDemoPage.jsx** +0 −62

```
@@ -1,62 +1,0 @@
1       - import { useState } from "react";
2       - import { AlertIcon, CheckIcon } from "../components/Icons";
3       -
4       - // Une page pour voir les 3 états d'un appel API : loading, succès, erreur
5       - export default function ApiDemoPage() {
6       -   const [loading, setLoading] = useState(false);
7       -   const [result, setResult] = useState(null); // { ok: true|false, message }
8       -   const [crash, setCrash] = useState(false);
9       -
10      -   // Pour tester l'Error Boundary : une erreur pendant le render
11      -   if (crash) {
12      -     throw new Error("Crash volontaire pour tester l'Error Boundary");
13      -   }
14      -
15      -   const simulate = (ok) => {
16      -     setResult(null);
17      -     setLoading(true);
18      -     setTimeout(() => {
19      -       setLoading(false);
20      -       setResult(ok ? { ok, message: "Data loaded successfully!" } : { ok, message: "Failed
        to load data. Please try again." });
21      -     }, 2000);
22      -   };
23      -
24      -   return (
25      -     <div className="container">
26      -       <h1 className="page-title">API Demo</h1>
27      -
28      -       <div className="card api-demo-card">
29      -         <p className="subtitle">Test loading states and error messages</p>
30      -
31      -         <div className="button-group">
32      -           <button onClick={() => simulate(true)} disabled={loading} className="btn btn-succ
        ess">
33      -             Simulate Success
34      -           </button>
35      -           <button onClick={() => simulate(false)} disabled={loading} className="btn btn-dan
        ger">
36      -             Simulate Error
37      -           </button>
38      -         </div>
```

```
-
-        <div className="button-group">
-          <button onClick={() => setCrash(true)} className="btn">
-            Crash the page (Error Boundary)
-          </button>
-        </div>
-
-        {loading && (
-          <div className="message-box loading-box">
-            <div className="spinner"></div>
-            <span>Loading...</span>
-          </div>
-        )}
-
-        {result && (
-          <div className={`message-box ${result.ok ? "success-box" : "error-box"}`}>
-            {result.ok ? <CheckIcon /> : <AlertIcon />}
-            <span>{result.message}</span>
-          </div>
-        )}
-      </div>
-    </div>
-  );
- }
```

SUPPRIMÉ **src/pages/CarDetailPage.jsx** +0 −89

```
@@ -1,89 +1,0 @@
- import { useQuery } from "@tanstack/react-query";
- import { useParams, Link } from "react-router-dom";
- import { useDispatch, useSelector } from "react-redux";
- import { fetchCarById } from "../api/actions";
- import { selectFavorites, toggleFavorite } from "../store/favoritesSlice";
- import { BackIcon, HeartIcon } from "../components/Icons";
- import Loading from "../components/Loading";
- import { getColorHex } from "../utils/colorUtils";
-
- export default function CarDetailPage() {
-   const { id } = useParams();
-   const dispatch = useDispatch();
-   const favorites = useSelector(selectFavorites);
-
-   // L'id fait partie de la clé : chaque voiture a son propre cache
-   const { data, isLoading, error } = useQuery({
-     queryKey: ["car", id],
-     queryFn: () => fetchCarById(id),
-   });
-
-   const car = data?.Car;
-   const isFavorite = car ? favorites.some((fav) => fav.id === car.id) : false;
-
-   if (isLoading) return <Loading text="Loading car details..." />;
-
-   if (error || !car) {
-     return (
-       <div className="container">
-         <p>Car not found</p>
-       </div>
-     );
-   }
-
-   return (
-     <div className="container">
-       <Link to="/" className="back-link">
-         <BackIcon />
```

```
38  -         Back to Catalog
39  -       </Link>
40  -
41  -       <div className="card detail-card">
42  -         <div className="detail-header">
43  -           <h1 className="detail-title">
44  -             {car.car} {car.car_model}
45  -           </h1>
46  -           <button className={`favorite-btn ${isFavorite ? "favorite-active" : ""}`} onClick
    ={() => dispatch(toggleFavorite(car))}>
47  -             <HeartIcon filled={isFavorite} />
48  -           </button>
49  -         </div>
50  -
51  -         <div className="detail-grid">
52  -           <div className="detail-item">
53  -             <span className="detail-label">Brand</span>
54  -             <span className="detail-value">{car.car}</span>
55  -           </div>
56  -           <div className="detail-item">
57  -             <span className="detail-label">Model</span>
58  -             <span className="detail-value">{car.car_model}</span>
59  -           </div>
60  -           <div className="detail-item">
61  -             <span className="detail-label">Color</span>
62  -             <div className="color-detail">
63  -               <div className="color-indicator" style={{ backgroundColor: getColorHex(car.ca
    r_color) }}></div>
64  -               <span className="detail-value">{car.car_color}</span>
65  -             </div>
66  -           </div>
67  -           <div className="detail-item">
68  -             <span className="detail-label">Year</span>
69  -             <span className="detail-value">{car.car_model_year}</span>
70  -           </div>
71  -           <div className="detail-item">
72  -             <span className="detail-label">VIN</span>
73  -             <span className="detail-value">{car.car_vin}</span>
74  -           </div>
75  -           <div className="detail-item">
76  -             <span className="detail-label">Price</span>
77  -             <span className="detail-value price">{car.price}</span>
78  -           </div>
79  -           <div className="detail-item">
80  -             <span className="detail-label">Availability</span>
81  -             <span className={`detail-value ${car.availability ? "status-available" : "statu
    s-unavailable"}`}>
82  -               {car.availability ? "Available" : "Unavailable"}
83  -             </span>
84  -           </div>
85  -         </div>
86  -       </div>
87  -     </div>
88  -   );
89  - }
```

SUPPRIMÉ **src/pages/FavoritesPage.jsx** +0 −23

```
@@ -1,23 +1,0 @@
1   - import { useDispatch, useSelector } from "react-redux";
2   - import CarCard from "../components/CarCard";
3   - import { selectFavorites, toggleFavorite } from "../store/favoritesSlice";
4   -
5   - export default function FavoritesPage() {
6   -   const dispatch = useDispatch();
```

```
-   const favorites = useSelector(selectFavorites);
-
-   return (
-     <div className="container">
-       <h1 className="page-title">My Favorites</h1>
-       {favorites.length === 0 ? (
-         <p className="subtitle">No favorites yet. Add some cars from the home page!</p>
-       ) : (
-         <div className="car-grid">
-           {favorites.map((car) => (
-             <CarCard key={car.id} car={car} isFavorite onToggleFavorite={(c) => dispatch(toggleFavorite(c))} />
-           ))}
-         </div>
-       )}
-     </div>
-   );
- }
```

SUPPRIMÉ **src/pages/HomePage.jsx** +0 −44

```
@@ -1,44 +1,0 @@
- import { useQuery } from "@tanstack/react-query";
- import { useDispatch, useSelector } from "react-redux";
- import { fetchCarsByYear } from "../api/actions";
- import { selectFavorites, toggleFavorite } from "../store/favoritesSlice";
- import CarCard from "../components/CarCard";
- import Loading from "../components/Loading";
-
- export default function HomePage() {
-   const dispatch = useDispatch();
-   const favorites = useSelector(selectFavorites);
-   const { data, isLoading, error } = useQuery({
-     queryKey: ["cars", 2012],
-     queryFn: () => fetchCarsByYear(2012),
-   });
-
-   if (isLoading) return <Loading text="Loading cars..." />;
-
-   if (error) {
-     return (
-       <div className="center-page">
-         <p className="error-text">Error loading cars: {error.message}</p>
-       </div>
-     );
-   }
-
-   // On limite l'affichage à 6 voitures
-   const cars = (data?.Cars || []).slice(0, 6);
-
-   return (
-     <div className="container">
-       <h1 className="page-title">Car Catalog</h1>
-       <div className="car-grid">
-         {cars.map((car) => (
-           <CarCard
-             key={car.id}
-             car={car}
-             isFavorite={favorites.some((fav) => fav.id === car.id)}
-             onToggleFavorite={(c) => dispatch(toggleFavorite(c))}
-           />
-         ))}
-       </div>
-     </div>
-   );
```

```
44 - }
```

SUPPRIMÉ **src/pages/LoginPage.jsx** +0 −59

```
@@ -1,59 +1,0 @@
1  - import { useMutation } from "@tanstack/react-query";
2  - import { useNavigate } from "react-router-dom";
3  - import { login } from "../api/actions";
4  - import { storage } from "../utils/storage";
5  -
6  - export default function LoginPage() {
7  -   const navigate = useNavigate();
8  -
9  -   const loginMutation = useMutation({
10 -     mutationFn: ({ userName, password }) => login(userName, password),
11 -     onSuccess: (response) => {
12 -       // On garde le token et l'utilisateur dans le localStorage
13 -       if (response.session) storage.setToken(response.session);
14 -       if (response.userDetails) {
15 -         storage.setUser(response.userDetails);
16 -         // On prévient le Header que l'utilisateur a changé
17 -         window.dispatchEvent(new CustomEvent("userUpdated"));
18 -       }
19 -       navigate("/");
20 -     },
21 -   });
22 -
23 -   const handleSubmit = (e) => {
24 -     e.preventDefault();
25 -     const formData = new FormData(e.target);
26 -     loginMutation.mutate({ userName: formData.get("userName"), password: formData.get("pass
   word") });
27 -   };
28 -
29 -   return (
30 -     <div className="login-container">
31 -       <div className="card login-card">
32 -         <h1 className="login-title">Welcome Back</h1>
33 -         <p className="subtitle">Sign in to your account</p>
34 -         <form onSubmit={handleSubmit} className="login-form">
35 -           {loginMutation.isError && (
36 -             <p className="error-text">
37 -               {loginMutation.error?.response?.data?.message || loginMutation.error?.message
   || "Login failed. Please try again."}
38 -             </p>
39 -           )}
40 -           <div className="form-group">
41 -             <label htmlFor="userName" className="form-label">
42 -               Username
43 -             </label>
44 -             <input type="text" id="userName" name="userName" className="form-input" placeho
   lder="Enter your username" required />
45 -           </div>
46 -           <div className="form-group">
47 -             <label htmlFor="password" className="form-label">
48 -               Password
49 -             </label>
50 -             <input type="password" id="password" name="password" className="form-input" pla
   ceholder="Enter your password" required />
51 -           </div>
52 -           <button type="submit" className="btn" disabled={loginMutation.isPending}>
53 -             {loginMutation.isPending ? "Signing In..." : "Sign In"}
54 -           </button>
55 -         </form>
56 -       </div>
```

```
57 -     </div>
58 -   );
59 - }
```

SUPPRIMÉ **src/pages/NotFoundPage.jsx** +0 −13

```
@@ -1,13 +1,0 @@
1  - import { Link } from "react-router-dom";
2  -
3  - export default function NotFoundPage() {
4  -   return (
5  -     <div className="center-page">
6  -       <h1 className="not-found-title">404</h1>
7  -       <p className="not-found-text">Page not found</p>
8  -       <Link to="/" className="btn">
9  -         Go Home
10 -       </Link>
11 -     </div>
12 -   );
13 - }
```

SUPPRIMÉ **src/store/favoritesSlice.js** +0 −28

```
@@ -1,28 +1,0 @@
1  - import { createSlice } from "@reduxjs/toolkit";
2  -
3  - const initialState = {
4  -   favorites: [],
5  - };
6  -
7  - const favoritesSlice = createSlice({
8  -   name: "favorites",
9  -   initialState,
10 -   reducers: {
11 -     // Si la voiture est déjà en favori on la retire, sinon on l'ajoute
12 -     toggleFavorite: (state, action) => {
13 -       const carExists = state.favorites.some((car) => car.id === action.payload.id);
14 -       if (carExists) {
15 -         state.favorites = state.favorites.filter((car) => car.id !== action.payload.id);
16 -       } else {
17 -         state.favorites.push(action.payload);
18 -       }
19 -     },
20 -   },
21 - });
22 -
23 - export const { toggleFavorite } = favoritesSlice.actions;
24 -
25 - // Selectors
26 - export const selectFavorites = (state) => state.favorites.favorites;
27 -
28 - export default favoritesSlice.reducer;
```

SUPPRIMÉ **src/store/favoritesSlice.test.js** +0 −62

```
@@ -1,62 +1,0 @@
1  - import { describe, it, expect } from 'vitest';
2  - import favoritesReducer, { toggleFavorite } from './favoritesSlice';
3  -
4  - describe('favoritesSlice', () => {
5  -   const mockCar = {
6  -     id: 1,
7  -     car: 'Tesla',
8  -     car_model: 'Model S',
9  -     car_color: 'Red',
```

```
-     car_model_year: 2024,
-     car_vin: 'TEST123',
-     price: '$79,990',
-     availability: true,
-   };
-
-   const mockCar2 = {
-     id: 2,
-     car: 'BMW',
-     car_model: 'i8',
-     car_color: 'Blue',
-     car_model_year: 2023,
-     car_vin: 'TEST456',
-     price: '$147,500',
-     availability: false,
-   };
-
-   it('should return initial state', () => {
-     expect(favoritesReducer(undefined, { type: 'unknown' })).toEqual({
-       favorites: [],
-     });
-   });
-
-   it('should add a car to favorites', () => {
-     const state = favoritesReducer(undefined, toggleFavorite(mockCar));
-     expect(state.favorites).toHaveLength(1);
-     expect(state.favorites[0]).toEqual(mockCar);
-   });
-
-   it('should remove a car from favorites when toggled again', () => {
-     const initialState = { favorites: [mockCar] };
-     const state = favoritesReducer(initialState, toggleFavorite(mockCar));
-     expect(state.favorites).toHaveLength(0);
-   });
-
-   it('should add multiple cars to favorites', () => {
-     let state = favoritesReducer(undefined, toggleFavorite(mockCar));
-     state = favoritesReducer(state, toggleFavorite(mockCar2));
-
-     expect(state.favorites).toHaveLength(2);
-     expect(state.favorites).toContainEqual(mockCar);
-     expect(state.favorites).toContainEqual(mockCar2);
-   });
-
-   it('should remove specific car without affecting others', () => {
-     const initialState = { favorites: [mockCar, mockCar2] };
-     const state = favoritesReducer(initialState, toggleFavorite(mockCar));
-
-     expect(state.favorites).toHaveLength(1);
-     expect(state.favorites[0]).toEqual(mockCar2);
-   });
- });
-
```

SUPPRIMÉ **src/store/store.js** +0 −8

```
@@ -1,8 +1,0 @@
- import { configureStore } from "@reduxjs/toolkit";
- import favoritesReducer from "./favoritesSlice";
-
- export const store = configureStore({
-   reducer: {
-     favorites: favoritesReducer,
-   },
- });
```

SUPPRIMÉ **src/utils/authLoader.js** +0 −19

```
@@ -1,19 +1,0 @@
- import { redirect } from "react-router-dom";
- import { storage } from "./storage";
-
- // Loader de /favorites : pas de token => redirection vers /login (avant même d'afficher la page)
- export const requireAuth = () => {
-   const token = storage.getToken();
-   if (!token) {
-     throw redirect("/login");
-   }
-   return { token };
- };
-
- // Loader de /login : déjà connecté => retour à l'accueil
- export const redirectIfAuthenticated = () => {
-   if (storage.getToken()) {
-     throw redirect("/");
-   }
-   return null;
- };
```

SUPPRIMÉ **src/utils/colorUtils.js** +0 −23

```
@@ -1,23 +1,0 @@
- // L'API donne un nom de couleur : on le transforme en code couleur pour la pastille.
- const colorMap = {
-   Red: "#e74c3c",
-   Blue: "#3498db",
-   Green: "#2ecc71",
-   Yellow: "#f1c40f",
-   Orange: "#f39c12",
-   Purple: "#9b59b6",
-   Pink: "#e91e63",
-   Violet: "#9b59b6",
-   Turquoise: "#1abc9c",
-   Crimson: "#dc143c",
-   Indigo: "#4b0082",
-   Mauv: "#e0b0ff",
-   Fuscia: "#ff00ff",
-   Khaki: "#f0e68c",
-   Maroon: "#800000",
-   Aquamarine: "#7fffd4",
-   Puce: "#cc8899",
-   Goldenrod: "#daa520",
- };
-
- export const getColorHex = (colorName) => colorMap[colorName] || "#95a5a6";
```

SUPPRIMÉ **src/utils/storage.js** +0 −16

```
@@ -1,16 +1,0 @@
- // Le token et l'utilisateur sont gardés dans le localStorage (la session survit au rechargement)
- const TOKEN_KEY = "auth_token";
- const USER_KEY = "auth_user";
-
- export const storage = {
-   setToken: (token) => localStorage.setItem(TOKEN_KEY, token),
-   getToken: () => localStorage.getItem(TOKEN_KEY),
-   removeToken: () => localStorage.removeItem(TOKEN_KEY),
-
-   setUser: (user) => localStorage.setItem(USER_KEY, JSON.stringify(user)),
```

```
11        -   getUser: () => {
12        -     const user = localStorage.getItem(USER_KEY);
13        -     return user ? JSON.parse(user) : null;
14        -   },
15        -   removeUser: () => localStorage.removeItem(USER_KEY),
16        - };
```

MODIFIÉ **vite.config.js** +7 −0

```
          @@ -1,8 +1,15 @@
1         + import { fileURLToPath, URL } from "node:url";
1   2       import { defineConfig } from "vite";
2   3       import react from "@vitejs/plugin-react";
3   4
4   5       export default defineConfig({
5   6         plugins: [react()],
7         +   // Alias : "@/features/cars/..." au lieu de "../../../features/cars/..."
8         +   resolve: {
9         +     alias: {
10        +       "@": fileURLToPath(new URL("./src", import.meta.url)),
11        +     },
12        +   },
6   13        server: {
7   14          // Le proxy évite les erreurs CORS en dev : /api/... est envoyé à https://myfakeapi.com/api/...
8   15          proxy: {
```

Étape 9 → 10 etape-9-architecture → etape-10-ia

L'IA : CLAUDE.md, .mcp.json (Playwright, Chrome DevTools, Figma), la skill react-feature, des exemples de prompts (voir IA.md)

AJOUTÉ `.claude/skills/react-feature/SKILL.md` +20 −0

```
@@ -1,0 +1,20 @@
+ ---
+ name: react-feature
+ description: Créer ou migrer un écran React dans une feature du projet Car Catalog (view, scene, adapter + test de l'adapter). À utiliser quand on demande un nouvel écran, une nouvelle page, ou de ranger une page existante dans l'architecture.
+ ---
+
+ # Créer un écran (View / Scene / Adapter)
+
+ 1. Trouver la feature (`src/features/<feature>/`). Si elle n'existe pas, la créer avec `api/`, `hooks/` et `screens/`.
+ 2. Créer le dossier `src/features/<feature>/screens/<X>Screen/` avec :
+    - `<X>Screen.view.jsx` : appelle les hooks de la feature (`hooks/`), gère loading (`Loading`), erreur (`ErrorMessage`) et vide, appelle l'adapter, passe le résultat à la Scene.
+    - `<X>Screen.adapter.js` : une ou plusieurs fonctions **pures** (pas de React, pas d'import d'API). Elles gèrent les valeurs manquantes (liste vide, « — », couleur par défaut). Réutiliser `features/cars/adapters/carAdapter.js` quand il s'agit de voitures.
+    - `<X>Screen.scene.jsx` : uniquement des props, assemble les composants de `@/components/...`. Pas de hook de données.
+    - `<X>Screen.adapter.test.js` : tests Vitest de l'adapter (cas normal, valeurs manquantes, réponse vide ou mal formée).
+    - `index.js` : `export { <X>ScreenView as <X>Screen } from "./<X>Screen.view";`
+ 3. Les appels HTTP vont dans `features/<feature>/api/` (avec `axiosQuery` de `@/lib`), les hooks React Query dans `features/<feature>/hooks/` (clés dans `queryKeys.js`).
+ 4. Si un composant d'UI manque, le créer dans le bon niveau de `src/components/` (atom, molecule, organism) avec des props simples, et l'exporter dans son `index.js`.
+ 5. Brancher l'écran dans `src/app/router.jsx` en important depuis `index.js` (ajouter un `loader` si la route est protégée : `requireAuth`).
+ 6. Vérifier : `npm test` puis `npm run build`. Les deux doivent passer avant de rendre la main.
+
+ Exemple de référence : `src/features/cars/screens/HomeScreen/`.
```

AJOUTÉ `.mcp.json` +16 −0

```
@@ -1,0 +1,16 @@
+ {
+   "mcpServers": {
+     "playwright": {
+       "command": "npx",
+       "args": ["@playwright/mcp@latest"]
+     },
+     "chrome-devtools": {
+       "command": "npx",
+       "args": ["chrome-devtools-mcp@latest"]
+     },
+     "figma-desktop": {
+       "type": "http",
+       "url": "http://127.0.0.1:3845/mcp"
+     }
+   }
+ }
```

AJOUTÉ `CLAUDE.md` +37 −0

```
@@ -1,0 +1,37 @@
+ # Car Catalog
+
+ Catalogue de voitures (formation React, jour 5).
+ Stack : React 19, Vite 7, React Router v7 (data router + loaders), React Query 5, Redux Toolkit, axios, Vitest, Playwright.
+ API : https://myfakeapi.com, appelée via le proxy Vite `/api/` (voir `vite.config.js`).
+
+ ## Commandes
+
+ - `npm run dev` : lancer l'application
+ - `npm test` : tests unitaires (Vitest, fichiers `src/**/*.test.js`)
+ - `npm run build` : build de production
+ - `npm run test:e2e` : tests end-to-end (Playwright, dossier `tests/e2e`)
+
+ ## Architecture (détails dans ARCHITECTURE.md)
+
+ - `app/` : router, providers, store, RootLayout
+ - `lib/` : axios, queryClient, config
+ - `components/` : UI partagée en atomic design (`atoms`, `molecules`, `organisms`, `templates`)
+ - `features/<feature>/` : le code métier (`api/`, `hooks/`, `adapters/`, `utils/`, `screens/`)
+ - Un écran = `features/<feature>/screens/<X>/`
+   - `X.view.jsx` : conteneur, appelle les hooks, gère loading / error / vide
+   - `X.adapter.js` : fonction pure, données de l'API -> modèle affiché
+   - `X.scene.jsx` : présentation, uniquement des props
+   - `X.adapter.test.js` : tests de l'adapter
+   - `index.js` : réexporte l'écran
+
+ ## Règles
+
+ - Toujours écrire les tests de l'adapter, puis lancer `npm test` et `npm run build`.
+ - Jamais d'appel API, de hook React Query ou de Redux dans une Scene ou dans `components/`.
+ - `components/` n'importe jamais `features/`.
+ - Imports avec l'alias `@` (vers `src/`) et via les `index.js`.
+ - Réutiliser les composants et les classes CSS existants (`src/styles.css`) avant d'en créer.
+ - Les clés React Query sont dans `features/<feature>/hooks/queryKeys.js`.
+ - Les favoris dans Redux restent les objets bruts de l'API.
+ - JavaScript (pas de TypeScript), composants en fonctions, textes de l'UI en anglais.
+ - Ne pas modifier `tests/e2e` sans le demander.
```

AJOUTÉ **IA.md** +29 −0

```
@@ -1,0 +1,29 @@
+ # Étape 10 : l'IA dans le projet
+
+ Même code que l'étape 9. On ajoute seulement ce qu'il faut pour travailler avec Claude Code :
+
+ | Fichier | Rôle |
+ |---|---|
+ | `CLAUDE.md` | Le contexte du projet, lu par Claude à chaque session : stack, commandes, architecture, règles. |
+ | `.mcp.json` | Les serveurs MCP du projet : Playwright (piloter un navigateur), Chrome Dev Tools (performance, console), Figma desktop (lire la maquette). |
+ | `.claude/skills/react-feature/SKILL.md` | Notre skill : créer ou migrer un écran en View / Scene / Adapter, avec le test de l'adapter. |
+ | `prompts/README.md` | Des exemples de prompts : Figma pixel perfect, Jira, skill, tests unitaires, e2e, performance. |
+
+ ## Mise en place
+
```

```bash
npm install
claude            # dans ce dossier : CLAUDE.md, .mcp.json et la skill sont chargés
/mcp              # vérifier les serveurs MCP
```

- Figma : ouvrir l'application Figma desktop, activer le serveur MCP Dev Mode (Preferences), sélectionner un frame.
- Jira : connecter le serveur MCP Atlassian (connecteur claude.ai, ou `claude mcp add` avec l'URL donnée dans la doc Atlassian).
- Skills existantes à montrer : front premium (design), navigateur, performance React, Argent (piloter un simulateur iOS / Android).

Le déroulé conseillé

1. On migre la Home ensemble, à la main, vers View / Scene / Adapter (étape 9).
2. On demande à Claude de migrer les autres écrans avec la skill `react-feature`.
3. On compare avec la correction de l'étape 9.
4. Les tests unitaires des adapters : écrits par Claude. Les parcours e2e : faits par Claude avec le MCP Playwright.